Machine Learning (2026) 115:239
https://doi.org/10.1007/s10994-026-07176-x

SURGE: Structural Perturbation Response Field Guided Residual Distillation for Graph Unlearning

Chaofan Shen[1] · **Mingyu Wang**[1] · **Jing Zhang**[1,2]

Received: 1 March 2026 / Revised: 18 August 2026 / Accepted: 20 September 2026
© The Author(s), under exclusive licence to Springer Science+Business Media LLC, part of Springer Nature 2026

Abstract

We introduce SURGE, a graph unlearning framework that treats deletion requests as prediction-level structural perturbations. Graph unlearning requires removing requested nodes, edges, or features from a trained graph neural network (GNN) without full retraining while preserving utility on retained data. Existing approaches either disrupt graph structure through partitioning, approximate removal in parameter space, or rely on fixed neighborhoods that cannot express heterogeneous node sensitivity. SURGE addresses this limitation with a Sense–Repair–Distill pipeline: it estimates a continuous response field over all nodes, solves graph-regularized corrected logits without backpropagation, and transfers the corrected knowledge through response-adaptive distillation. Across seven standard benchmarks and multiple GNN backbones, SURGE achieves leading or competitive utility and forgetting effectiveness for node, edge, and feature unlearning. In a matched 18-cell evaluation across deletion types, datasets, backbones, and ten random seeds, SURGE attains the highest mean micro-F1 (0.8632) and retraining faithfulness (0.0243 test JSD) among SURGE, ETR, MEGU, and IDEA; an ogbn-arxiv ablation further supports scalability on a large graph.

Keywords Graph unlearning · Graph neural networks · Knowledge distillation · Machine unlearning · Privacy

Editor: Quanming Yao.

✉ Jing Zhang
jingz@seu.edu.cn

Chaofan Shen
chaofans@seu.edu.cn

Mingyu Wang
wangmingyu@seu.edu.cn

[1] School of Cyber Science and Engineering, Southeast University, No. 2 SEU Road, Nanjing 211189, China

[2] Engineering Research Center of Blockchain Application, Supervision and Management (Southeast University), Ministry of Education, No. 2 SEU Road, Nanjing 211189, China

Published online: 07 October 2026

1 Introduction

Graph unlearning asks whether a deployed graph neural network (GNN) can remove request-specific information from nodes, edges, or features while retaining predictive utility on the remaining graph. This problem extends machine unlearning, whose goal is to eliminate the influence of target samples without retraining from scratch (Xu et al., 2024; Nguyen et al., 2025; Bourtoule et al., 2021; Ginart et al., 2019; Cao, 2015). The capability is important for privacy regulations such as the General Data Protection Regulation (GDPR) and the California Consumer Privacy Act (CCPA), which grant individuals the right to request deletion of personal data (Gdpr-Info.Eu, 2016; California Department of Justice, 2018). In graph applications, the same requirement arises when users request deletion in social networks, or when noisy edges and mislabeled nodes must be removed to maintain model robustness.

In graphs, data removal is also a reliability problem. GNNs learn node representations through iterative message passing and neighborhood aggregation (Kipf & Welling, 2017; Veličković et al., 2018; Hamilton et al., 2017; Xu et al., 2019; Wu et al., 2021), so deleting even a single node or edge can propagate through multiple hops and change many other node representations. From the broader trustworthy GNN perspective, graph unlearning connects privacy, robustness, and adaptive structural sensitivity modeling (Dai et al., 2024). A useful unlearning method should therefore identify not only what is deleted, but also where the trained model's predictions react to the deletion. This structural coupling renders conventional machine unlearning methods insufficient for graph-structured data (Cheng et al., 2023; Wu et al., 2023), and graph unlearning has therefore emerged as a specialized research direction (Chen et al., 2022; Said et al., 2023).

Existing graph unlearning methods can be broadly organized into three paradigms, each with distinct strengths but lacking a direct, continuous, prediction-level signal for request-dependent correction routing. Partition-based methods, such as GraphEraser (Chen et al., 2022) and GUIDE (Wang et al., 2023), split the training graph into disjoint shards and retrain only the affected submodels upon a deletion request. While computationally efficient, graph partitioning inevitably severs important edges across shards, leading to significant utility degradation on graphs with strong connectivity. Moreover, treating shards as independent units entirely bypasses the question of how individual nodes are affected by the removal. Influence function methods, such as GIF (Wu et al., 2023), CGU (Chien et al., 2022), ETR (Yang et al., 2025), and IDEA (Dong et al., 2024), approximate the effect of data removal through parameter corrections or editing based on importance estimates. These methods avoid the utility loss caused by partitioning but rely on local curvature approximations that may become inaccurate under large distributional shifts. Although they provide sensitivity estimates at the parameter level, they do not directly quantify how individual node predictions are affected. Learning-based methods, such as GNNDelete (Cheng et al., 2023) and MEGU (Li et al., 2024), employ auxiliary modules or specially designed loss functions to jointly optimize forgetting and retention. These methods typically rely on predefined neighborhoods of fixed size to identify the influenced region, treating topological proximity as a coarse proxy for influence. They also often require extra training on the remaining data, incurring considerable computational overhead. In summary, prior paradigms either operate on coarse graph partitions, approximate parameter-space effects, or use fixed neighborhoods as the correction region. They do not directly expose how strongly each node's current prediction reacts to a given request. These realized responses can vary with topology, features,

and aggregation patterns, so a fixed-radius mask may be either too narrow or unnecessarily broad. This motivates a continuous correction-routing signal while leaving exact training influence as a separate question.

Removing an entity from a graph exposes a realized prediction-sensitivity signal under the trained model. A large output shift indicates that a node's current prediction reacts to the request, but it does not identify retraining influence or provide an exact affected-region label. This signal complements parameter-space influence estimates and neighborhood reachability without requiring second-order computation or a manually fixed boundary. Building upon this insight, we propose **SURGE** (**S**tructural Perturbation Response Field G**u**ided **R**esidual Distillation for **G**raph Unl**e**arning), a framework comprising three stages—Sense, Repair, Distill—for effective and efficient graph unlearning. In the Sense stage, SURGE constructs a structural perturbation response field that assigns a continuous prediction-sensitivity score to each node. In the Repair stage, the response field guides a solver with graph regularization to generate corrected teacher logits without backpropagation. In the Distill stage, adaptive knowledge distillation transfers the corrected knowledge into the model, emphasizing nodes that receive the largest corrected-teacher updates while encouraging stability on low-response nodes. The entire framework handles node, edge, and feature unlearning through a unified pipeline, with the response field adapting to each scenario. SURGE uses this request-conditioned response score to route logit correction, while distillation weights are determined by the magnitude of the corrected-teacher update. Overall, the main contributions of this work are as follows:

- We introduce the structural perturbation response field as a continuous, request-dependent prediction-sensitivity signal for graph unlearning. The field is theoretically motivated under the linear-GNN regime and serves as a topology-adaptive proxy for routing unlearning corrections rather than an exact localization or training-influence ranking.
- Building on the response field, we develop a residual correction strategy that generates corrected predictions through a solver with graph regularization, requiring no backpropagation. The response field simultaneously modulates message propagation and regularization strength, enabling adaptively calibrated corrections for heterogeneously affected nodes at low computational cost.
- Integrating these components, we present SURGE, a unified framework that handles node, edge, and feature unlearning through a single pipeline. Experiments on seven standard benchmarks with multiple GNN backbones demonstrate leading or competitive utility and forgetting effectiveness. A controlled node/edge/feature evaluation shows the strongest macro-average utility and retraining-faithfulness among SURGE, ETR, MEGU, and IDEA, and an ogbn-arxiv ablation further tests scalability on a large graph.

2 Related Work

2.1 Machine Unlearning

Machine unlearning aims to remove the influence of specific training data from a learned model so that the resulting model behaves as if the data had never been used (Cao, 2015; Bourtoule et al., 2021; Xu et al., 2024; Nguyen et al., 2025). Existing methods fall into two

broad categories. Exact unlearning produces a model statistically indistinguishable from one retrained from scratch, while approximate unlearning relaxes this requirement for computational efficiency. Recent approximate approaches have explored certified removal via Newton updates (Guo et al., 2020), data partitioning for efficient retraining (Bourtoule et al., 2021), and parameter corrections based on influence functions (Izzo et al., 2021; Sekhari et al., 2021). More recent work has investigated weight saliency for selective forgetting (Fan et al., 2024), model sparsification to reduce unlearning complexity (Jia et al., 2023), and synaptic dampening for rapid parameter adjustment without retraining (Foster et al., 2024). These methods primarily operate in the parameter space, relying on gradient information or curvature estimates to approximate the effect of removal. SURGE differs by performing corrections in the output space through response analysis at the prediction level, bypassing approximations in the parameter space entirely.

2.2 Graph Unlearning

Graph unlearning extends machine unlearning to GNNs (Said et al., 2023). The message passing mechanism in GNNs creates dependencies between nodes, propagating the influence of each data point across multiple hops (Chen et al., 2022; Wu et al., 2023). Removing even a single node or edge can therefore affect the representations of many distant nodes, making conventional unlearning methods insufficient for graphs. Recent methods address this challenge through different strategies but share common limitations. Table 1 summarizes these representative paradigms, their practical limitations, and the corresponding contrast with SURGE. Partition-based approaches such as GraphEraser (Chen et al., 2022) and GUIDE (Wang et al., 2023) split the graph into disjoint shards and retrain only

Table 1 Representative graph unlearning paradigms, their practical limitations, and how SURGE addresses each limitation

Paradigm	Typical mechanism	Practical limitation	SURGE contrast
Partitioning	Shard the graph and retrain affected submodels	Can break cross-shard dependencies and degrade utility on connected graphs	Keeps the graph intact and corrects predictions on the post-unlearning graph
Parameter editing	Approximate parameter changes with curvature or importance estimates	Relies on local approximations and often requires backward passes or parameter-space estimates	Uses forward prediction shifts and output-space correction before lightweight distillation
Learning-based	Optimize auxiliary deletion or retention objectives	Often depends on fixed neighborhoods or extra trainable modules	Uses a continuous response field to modulate correction and distillation per node
SURGE	Sense prediction shifts, repair logits, and distill adaptively	Uses cached or recomputed original logits as the deployment reference	Provides request-dependent correction routing without fixed-radius boundaries

affected submodels. However, partitioning inevitably severs structural connections and degrades utility, especially on densely connected graphs. Projection-based methods such as Cong and Mahdavi (2023) achieve efficient forgetting by projecting model parameters onto subspaces that are approximately orthogonal to the influence of the forgotten data, yet their effectiveness depends on the quality of the subspace approximation and may degrade for complex graph topologies. ETR (Yang et al., 2025) erases and rectifies model parameters using importance estimates without fine-tuning, yet its corrections rely on local approximations that may become inaccurate under large structural changes. IDEA (Dong et al., 2024) achieves certified unlearning through parameter modifications based on Hessian approximations, but the computational cost limits its scalability. GNNDelete (Cheng et al., 2023) modifies learned representations using deletion operators guided by edge consistency and neighborhood influence constraints, but depends on predefined neighborhoods that do not adapt to heterogeneous topology. MEGU (Li et al., 2024) jointly optimizes forgetting and retention through a mutual evolution mechanism but still relies on neighborhoods of fixed size to determine the scope of influence.

Across these approaches, a common limitation is the absence of an adaptive node-level signal for routing corrections. Most existing methods require comparable or stronger data access than SURGE at unlearning time: GIF and IDEA compute Hessian or influence-function quantities via backward passes on the original graph; ETR estimates parameter importance from the original model; and partition-based methods retain independently trained shard models derived from $\mathcal{G}$. SURGE constructs this routing signal from prediction sensitivity before and after the graph modification. The response field adapts to heterogeneous topology and aggregation patterns without graph partitioning, curvature estimation, or predefined neighborhood boundaries, but it is not treated as an exact influence label.

2.3 Knowledge Distillation

Knowledge distillation (Hinton et al., 2015; Gou et al., 2021) transfers knowledge from a teacher to a student model by matching soft output distributions. Recent work has explored distillation for machine unlearning outside the graph domain. SCRUB (Kurmanji et al., 2023) trains a student to match the teacher on retained data while diverging on forgotten data. Chundawat et al. (2023) guide forgetting by providing incorrect soft labels as teaching signals. While these works demonstrate the potential of distillation for forgetting in the i.i.d. data setting, applying distillation to graph unlearning—where structural dependencies must be accounted for—has received comparatively limited attention. SURGE introduces a distinct formulation in which the teacher is not a separately trained model but is constructed through logit correction guided by the response field. Distillation weights are determined by the magnitude of correction at each node, emphasizing predictions that the repair stage changes most while preserving low-update predictions elsewhere.

2.4 Reliability-Driven Adaptive Modeling

SURGE also relates to reliability-driven learning, where models adapt their behavior using uncertainty, robustness, or sensitivity signals. Recent work has explored this principle in multimodal affect perception, multimedia information fusion, federated uncertainty quantification, privacy-preserving federated learning, and Bayesian optimization (Zhang et al.,

2025; Zhu et al., 2025; Wang et al., 2025; Li et al., 2026; Chen et al., 2026a; Zhu et al., 2026; Chen et al., 2025; Tian et al., 2026; Chen et al., 2026b). In graph unlearning, the relevant reliability signal is request-conditioned structural response: the model should identify which predictions change when the graph is edited and route correction accordingly. SURGE instantiates this principle in graph unlearning through response sensing, graph-regularized repair, and response-adaptive distillation.

3 Preliminaries

3.1 Notation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathrm{X})$ denote an attributed graph, where $\mathcal{V}$ is the set of n nodes, $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges, and $\mathrm{X} \in \mathbb{R}^{n \times f}$ is the node feature matrix. Let $\mathrm{A} \in \{0,1\}^{n \times n}$ be the adjacency matrix and $\hat{\mathrm{A}}$ its normalized variant. For semi-supervised node classification, each node i in the labeled set $\mathcal{V}_L \subset \mathcal{V}$ is associated with a label $y_i \in \{1, \ldots, c\}$, where c denotes the number of classes.

3.2 Graph Neural Networks

A GNN learns node representations by iteratively aggregating information from neighborhoods (Hamilton et al., 2017; Kipf & Welling, 2017; Veličković et al., 2018; Wu et al., 2021; Xu et al., 2019). In the l-th layer of a general message-passing GNN, each node i first collects messages from its neighbors and then updates its own representation:

$$\mathrm{h}_i^{(l)} = \phi^{(l)}\Big(\mathrm{h}_i^{(l-1)}, \bigoplus_{j \in \mathcal{N}(i)} \psi^{(l)}(\mathrm{h}_i^{(l-1)}, \mathrm{h}_j^{(l-1)})\Big), \tag{1}$$

where $\mathcal{N}(i)$ denotes the neighbors of node i, $\psi^{(l)}$ is the message function that computes the message sent from node j to node i, $\bigoplus$ denotes a permutation-invariant aggregation operator (e.g., sum, mean, or max), $\phi^{(l)}$ is the update function, and $\mathrm{h}_i^{(0)} = \mathrm{X}_i$. After L layers, the GNN f_θ parameterized by θ produces logit outputs $\mathrm{Z} = f_\theta(\mathcal{G}) \in \mathbb{R}^{n \times c}$.

3.3 Graph Unlearning

Given a GNN f_θ with optimal parameters θ^* trained on graph $\mathcal{G}$, a graph unlearning request specifies the entities whose influence should be removed from the trained model. Applying the request produces the post-unlearning graph $\mathcal{G}^-$ (with edge set $\mathcal{E}^-$). The objective of graph unlearning is to efficiently derive an updated model $f_{\theta'}$ whose behavior closely approximates that of a model retrained from scratch on $\mathcal{G}^-$. Depending on the type of entities to be forgotten, graph unlearning can be categorized into three tasks:

- **Node Unlearning:** A subset of training nodes $\Delta\mathcal{V} \subset \mathcal{V}_L$ is designated as unlearning targets. All edges incident to nodes in $\Delta\mathcal{V}$ are removed from the graph, and these nodes are excluded from the labeled training set used for subsequent model updates.
- **Edge Unlearning:** A set of edges $\Delta\mathcal{E} \subset \mathcal{E}$ is designated as unlearning targets and re-

moved from the graph.
- **Feature Unlearning:** For a subset of training nodes, designated feature dimensions are erased (replaced with zeros) in $\mathcal{G}^-$.

We use node unlearning as the primary setting to illustrate the proposed method. The framework handles edge and feature unlearning through the same pipeline, with detailed demonstrations provided in the experimental section.

3.4 Threat Model and Information Access

We make explicit the information available to SURGE at unlearning time. Upon receiving a request, the algorithm has access to: (i) the trained model f_θ and its parameters; (ii) the post-unlearning graph $\mathcal{G}^-$, together with the labels of retained training nodes used by the supervised term; (iii) the specification of the unlearning request (i.e., which entities are to be forgotten); and (iv) cached original logits $\mathbf{Z} = f_\theta(\mathcal{G})$. The original graph $\mathcal{G}$ itself is not required at request time when this cache is available. Because $\mathbf{Z}$ depends only on the trained model and the graph available at training time, it can be precomputed and cached immediately after training, before any unlearning request arrives. The cache contains an $n \times c$ logit matrix rather than raw features, labels, or edges, and can be stored with the trained model as deployment metadata. Under this caching strategy, processing a deletion request does not require access to the pre-deletion training graph or to the raw data of forgotten entities; it uses the cached logits, $\mathcal{G}^-$, and retained supervision. If the cache is unavailable, an implementation may recompute $\mathbf{Z}$ through a single read-only forward pass on $\mathcal{G}$ before deletion or in a protected training environment. This fallback assumes stronger data availability than the cached deployment path, but still performs no gradient computation on the original graph. We note that comparable or stronger data access assumptions are standard among existing methods, as discussed in Sect. 2.2. SURGE requires strictly less computation on the original graph than methods that perform Hessian or influence function evaluations via backward passes.

For feature unlearning, the request modifies the feature matrix rather than the adjacency. The same response definition remains applicable because message passing mixes transformed node features through the graph; erasing features at target nodes can therefore change both their own predictions and predictions of neighboring nodes after aggregation. The structural interpretation is weaker than in edge or node deletion, and we treat the response field as an output-space perturbation proxy in this case rather than as direct evidence of structural influence.

4 Methodology

4.1 Framework Overview

The overall framework of SURGE is depicted in Fig. 1. For effective and efficient graph unlearning, SURGE adopts a three-stage pipeline comprising Sense, Repair, and Distill. The pipeline begins by constructing a structural perturbation response field, which compares cached original output distributions with the GNN's output distributions on the post-

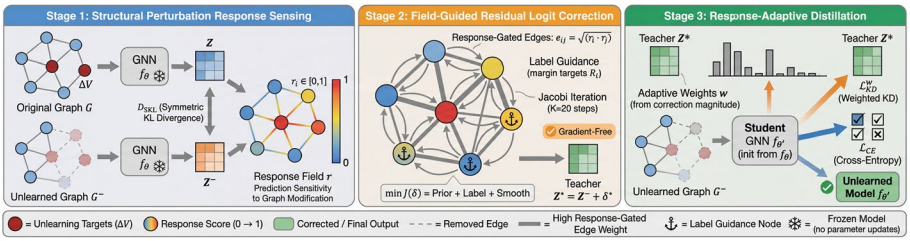


Fig. 1 Overview of SURGE for graph unlearning. Prediction shifts induced by a deletion request define a structural response field, which then guides residual logit correction and response-adaptive distillation

unlearning graph $\mathcal{G}^-$ to produce a continuous response score for each node, quantifying how strongly its prediction reacts to the deletion request under message passing. This response field then drives the Repair stage, where field-guided residual logit correction generates corrected teacher logits via optimization with graph regularization: the field simultaneously modulates message passing along edges and regularization at individual nodes, allowing larger changes in high-response portions of the graph while preserving low-response predictions. Finally, the corrected logits are transferred into a lightweight student model through response-adaptive knowledge distillation, in which the distillation weight of each node is determined by the magnitude of logit correction. The Sense and Repair stages are gradient-free, requiring only forward passes on the post-unlearning graph and sparse linear iterations without backpropagation, which significantly reduces computational overhead.

The three stages use the response signal in complementary ways. The scalar r_i identifies how much node i reacts to the request. In the Repair stage, r_i induces node-level prior weights $w_i^p = (1 - r_i)^\kappa + \alpha$ and edge gates $e_{ij} = \sqrt{r_i r_j} + \varepsilon$, so high-response nodes can receive larger corrections and high-response neighborhoods exchange stronger correction signals. In the Distill stage, the final training weights are computed from the magnitude of the corrected teacher update rather than directly from r_i, ensuring that the student focuses on nodes for which the repair stage actually changed the teacher prediction.

The design separates three roles for the response signal. The linear-GNN analysis gives a one-way connection between nonzero output shifts and changed aggregation rows. The graph-regularized repair objective uses the response values to construct a corrected teacher in output space, and the Distill stage transfers that teacher into model parameters using correction-magnitude weights. These roles make the response field a correction-routing signal; they do not require each normalization, override, or weighting choice to be individually optimal.

4.2 Structural Perturbation Response Field

Rather than relying on fixed neighborhood boundaries or expensive influence estimation, we directly measure each node's prediction sensitivity to the graph modification by comparing the model's output distributions before and after applying the unlearning request, thereby providing a request-conditioned signal for routing corrections.

Specifically, given the trained GNN f_θ and cached original logits, we compute only the post-unlearning logits at request time:

$$\mathrm{Z} = f_\theta(\mathcal{G}) \text{ (cached)}, \quad \mathrm{Z}^- = f_\theta(\mathcal{G}^-). \tag{2}$$

The key observation is that if a graph modification causes a significant shift in a node's predictive distribution, that node incorporated substantial information from the affected entities during message passing. We formalize this intuition as the structural perturbation response score.

Definition 1 (Structural Perturbation Response Score) For logit vectors $\mathrm{z}_1, \mathrm{z}_2 \in \mathbb{R}^c$ and temperature T, define the symmetric KL divergence with temperature scaling (Hinton et al., 2015):

$$D_{\mathrm{SKL}}^T(\mathrm{z}_1, \mathrm{z}_2) = \frac{1}{2}\big[D_{\mathrm{KL}}\big(\sigma_T(\mathrm{z}_1)\|\sigma_T(\mathrm{z}_2)\big) + D_{\mathrm{KL}}\big(\sigma_T(\mathrm{z}_2)\|\sigma_T(\mathrm{z}_1)\big)\big], \tag{3}$$

where $\sigma_T(\mathrm{z}) = \mathrm{softmax}(\mathrm{z}/T)$. The response score of each node is then:

$$d_i = D_{\mathrm{SKL}}^{T_s}(\mathrm{Z}_i, \mathrm{Z}_i^-), \qquad r_i = \min\Big(\frac{d_i}{Q_\tau(\mathrm{d})}, 1\Big), \tag{4}$$

where $Q_\tau(\mathrm{d})$ is the τ-quantile of d, serving as a robust normalization reference. For node unlearning, unlearning targets in $\Delta\mathcal{V}$ are assigned $r_i = 1$, ensuring they receive maximal correction priority in subsequent stages regardless of the observed prediction shift.

Proposition 1 *(Output shift under linear GNN) For a single-layer linear GNN* $f(\mathrm{X}, \mathrm{A}) = \hat{\mathrm{A}}\mathrm{X}\mathrm{W}$*, consider an adjacency-only perturbation with fixed* X *and* W*. The output shift at node i induced by replacing* A *with the post-unlearning adjacency* A^- *is*

$$\Delta\mathrm{Z}_i = (\hat{\mathrm{A}}_i - \hat{\mathrm{A}}_i^-)\mathrm{X}\mathrm{W}, \tag{5}$$

If the normalized aggregation row is unchanged, then $\Delta\mathrm{Z}_i = 0$*. Equivalently,* $\Delta\mathrm{Z}_i \neq 0$ *implies that the normalized aggregation row has changed. The reverse implication need not hold because multiplication by* XW *can cancel a nonzero row change.*

Proof By linearity, $\Delta\mathrm{Z}_i = f(\mathrm{X}, \mathrm{A})_i - f(\mathrm{X}, \mathrm{A}^-)_i = (\hat{\mathrm{A}}_i - \hat{\mathrm{A}}_i^-)\mathrm{X}\mathrm{W}$. If $\hat{\mathrm{A}}_i = \hat{\mathrm{A}}_i^-$, the row difference is zero and therefore $\Delta\mathrm{Z}_i = 0$. Taking the contrapositive gives $\Delta\mathrm{Z}_i \neq 0 \Rightarrow \hat{\mathrm{A}}_i \neq \hat{\mathrm{A}}_i^-$. A nonzero row difference can nevertheless lie in a null space induced by XW or contain terms that cancel. □

Remark 1 *(Prediction sensitivity as a proxy for training influence)* The score r_i measures the prediction sensitivity of the trained model to the graph modification, whereas training influence quantifies how optimal parameters change upon reoptimization. SURGE uses prediction sensitivity as a computationally efficient proxy for routing where corrections should be considered. Proposition 1 provides one-way support in the linear regime: a nonzero $\Delta\mathrm{Z}_i$ certifies that the normalized aggregation row changed, but a changed row can still yield zero output shift through cancellation. Under local first-order views of retraining, these output shifts indicate regions where reoptimization may need to adjust the model's behavior, but they do not provide an exact ranking of parameter influence. For general nonlinear GNNs

with L layers, including attention-based architectures such as GAT, the response score measures the realized output effect under the trained model. It can therefore capture multi-hop and attention-modulated dependency patterns without requiring second-order computation, while remaining an imperfect correction-routing proxy.

This design offers three practical advantages. First, quantile normalization provides robustness to outliers and ensures comparable response scores across different graphs and unlearning requests. Second, unlike approaches that rely on fixed neighborhoods, the response field naturally adapts to the graph topology and the model's learned representations, capturing dependency propagation patterns that may extend beyond or fall short of a prescribed k-hop radius. Third, the continuous scores $r_i \in [0, 1]$ enable granular downstream modulation, as opposed to binary membership decisions.

4.3 Field-Guided Residual Logit Correction

Having quantified the perturbation response, we next generate corrected teacher logits Z^* that approximate the desired post-unlearning behavior. To avoid the catastrophic degradation associated with direct parameter modification, we instead operate in the logit space through a graph-regularized residual correction mechanism guided by the response field.

4.3.1 Optimization Formulation

We seek a residual correction $\boldsymbol{\delta}^* \in \mathbb{R}^{n \times c}$ such that $\mathrm{Z}^* = \mathrm{Z}^- + \boldsymbol{\delta}^*$ minimizes:

$$\mathcal{J}(\boldsymbol{\delta}) = \underbrace{\sum_i w_i^p \|\boldsymbol{\delta}_i\|^2}_{\text{Prior Regularization}} + \underbrace{\mu \sum_{i \in \mathcal{V}_L} s_i \|\boldsymbol{\delta}_i - \mathrm{R}_i\|^2}_{\text{Label Guidance}} + \underbrace{\lambda \sum_{(i,j) \in \mathcal{E}^-} e_{ij} \|\boldsymbol{\delta}_i - \boldsymbol{\delta}_j\|^2}_{\text{Structural Smoothness}}. \tag{6}$$

Each term serves a complementary purpose:

Prior Regularization. The weight $w_i^p = (1 - r_i)^\kappa + \alpha$ penalizes large corrections for low-response nodes, encouraging their original predictions to remain intact. Here $\alpha > 0$ is a small floor constant that keeps the weight strictly positive everywhere. Nodes with high response scores ($r_i \approx 1$) receive near-zero prior penalties, permitting substantial correction.

Label Guidance. For retained training nodes $i \in \mathcal{V}_L$, the target R_i encodes a soft margin correction with strength m: $R_{i,k} = m$ if $k = y_i$ and $R_{i,k} = -m/(c-1)$ otherwise. Note that $\sum_k R_{i,k} = 0$, so R_i adjusts only the relative class margins without shifting the overall logit scale. The supervision weight $s_i = r_i \cdot \mathbb{1}_{i \in \mathcal{V}_L}$ makes label guidance strongest for high-response training nodes.

Structural Smoothness. The graph-based term encourages neighboring nodes to receive coherent corrections. The edge weight $e_{ij} = \sqrt{r_i \cdot r_j + \varepsilon}$ strengthens smoothness coupling between high-response nodes and limits propagation through low-response portions of the graph.

4.3.2 Gradient-Free Iterative Solver

The objective $\mathcal{J}$ is a strictly convex quadratic: setting $\nabla_{\delta_i}\mathcal{J} = 0$ yields a sparse symmetric positive-definite linear system $\mathrm{H}\boldsymbol{\delta} = \mathrm{b}$. We solve it via Jacobi iteration initialized at $\boldsymbol{\delta}^{(0)} = 0$:

$$\boldsymbol{\delta}_i^{(t+1)} = \frac{\mu\, s_i\, \mathrm{R}_i + \lambda \sum_{j\in\mathcal{N}(i)} e_{ij}\, \boldsymbol{\delta}_j^{(t)}}{w_i^p + \mu\, s_i + \lambda\, \tilde{d}_i}, \tag{7}$$

where $\tilde{d}_i = \sum_{j\in\mathcal{N}(i)} e_{ij}$ is the degree weighted by the response scores. The corrected teacher logits are:

$$\mathrm{Z}^* = \mathrm{Z}^- + \boldsymbol{\delta}^{(K)}. \tag{8}$$

4.3.3 Convergence Analysis

For completeness, we verify that the optimization in Eq. (6) is well-posed and the iterative solver converges. Strict convexity and Jacobi convergence are standard consequences of diagonal dominance (Saad, 2003); we include the proof for completeness and to make explicit the role of the algorithmic parameters. The design contribution of the response-gated formulation lies not in the convergence guarantee itself, but in its control over the magnitude and direction of the correction at each node, as elaborated in Remark 2 below.

Proposition 2 *The objective $\mathcal{J}(\boldsymbol{\delta})$ in Eq. (6) is strictly convex with a unique global minimizer $\boldsymbol{\delta}^*$. The iterative update in Eq. (7), initialized at $\boldsymbol{\delta}^{(0)} = 0$, converges to $\boldsymbol{\delta}^*$ at a geometric rate.*

Proof $\mathcal{J}$ decomposes across the c class dimensions independently. For any single class dimension, it takes the quadratic form $\mathcal{J}_k(\boldsymbol{\delta}_k) = \boldsymbol{\delta}_k^\top \mathrm{H}\boldsymbol{\delta}_k - 2\mathrm{b}_k^\top \boldsymbol{\delta}_k + \text{const}$, where $\mathrm{H} \in \mathbb{R}^{n\times n}$ has entries

$$\begin{aligned} H_{ii} &= w_i^p + \mu s_i + \lambda \tilde{d}_i, \\ H_{ij} &= -\lambda e_{ij} \quad \text{for } (i,j) \in \mathcal{E}^-, \\ H_{ij} &= 0 \quad \text{otherwise.} \end{aligned} \tag{9}$$

Since $w_i^p = (1 - r_i)^\kappa + \alpha \geq \alpha > 0$, we have $H_{ii} > \lambda \tilde{d}_i = \sum_{j\neq i} |H_{ij}|$. Hence H is strictly diagonally dominant with positive diagonal, and by the Levy–Desplanques theorem, H is positive definite. Strict convexity and uniqueness of $\boldsymbol{\delta}^*$ follow immediately.

The update in Eq. (7) is the Jacobi iteration $\boldsymbol{\delta}^{(t+1)} = \mathrm{D}^{-1}(\mathrm{b} + \mathrm{L}\boldsymbol{\delta}^{(t)})$ for the system $\mathrm{H}\boldsymbol{\delta} = \mathrm{b}$, where $\mathrm{D} = \mathrm{diag}(\mathrm{H})$ and $\mathrm{L} = \mathrm{D} - \mathrm{H}$. The spectral radius satisfies

$$\rho(\mathrm{D}^{-1}\mathrm{L}) \leq \|\mathrm{D}^{-1}\mathrm{L}\|_\infty = \max_i \frac{\sum_{j\neq i} |H_{ij}|}{H_{ii}} = \max_i \frac{\lambda \tilde{d}_i}{w_i^p + \mu s_i + \lambda \tilde{d}_i} < 1, \tag{10}$$

where the last inequality follows from $w_i^p \geq \alpha > 0$. Hence the iteration converges geometrically. □

Remark 2 *(Response-adaptive correction magnitude)* The response field controls the correction magnitude at each node through the prior weight $w_i^p = (1 - r_i)^\kappa + \alpha$. For a low-response node ($r_i \approx 0$), $w_i^p \approx 1 + \alpha$ dominates the denominator in Eq. (7), suppressing $\|\boldsymbol{\delta}_i^*\|$ toward zero and preserving the original prediction. Conversely, for a high-response node ($r_i \approx 1$), $w_i^p \approx \alpha$ provides minimal resistance, allowing the label guidance term (for training nodes) or the neighbor-propagated correction (for non-training nodes) to drive a substantial update. Concurrently, the edge gates $e_{ij} = \sqrt{r_i \cdot r_j + \varepsilon}$ couple corrections more strongly within high-response portions of the graph while limiting propagation through low-response portions. This adaptive modulation of correction magnitude—independent of the standard convergence guarantee—is the primary design contribution of the response-gated formulation.

Practical convergence. The contraction factor of the Jacobi iteration is bounded by $\rho \leq \max_i \lambda \tilde{d}_i/(\alpha + \lambda \tilde{d}_i)$, which is controlled primarily by the prior floor α: larger α yields faster convergence at the cost of more conservative corrections. For our default setting ($\alpha = 0.05$, $\lambda = 0.05$), the bound evaluates to $\rho < 0.98$ on all benchmarks. Because this bound is pessimistic, convergence in practice is faster: we fix $K = 20$ iterations, which empirically yields a relative residual $\|\boldsymbol{\delta}^{(K)} - \boldsymbol{\delta}^*\|/\|\boldsymbol{\delta}^*\| < 10^{-3}$ across all benchmarks. The entire solve is gradient-free, with each iteration requiring only sparse aggregation over $\mathcal{E}^-$ at a cost of $O(|\mathcal{E}^-| \cdot c)$.

4.4 Response-Adaptive Knowledge Distillation

The teacher logits $\mathbf{Z}^*$ encode the corrected knowledge after unlearning in the logit space. To transfer this knowledge into the model's parameter space, we employ knowledge distillation with response-adaptive weighting of individual nodes.

4.4.1 Adaptive Weight Computation

Not all nodes require equal guidance during distillation. We compute a distillation weight for each node reflecting the magnitude of logit correction:

$$w_i = \min\Big(\frac{D_{\mathrm{SKL}}^{T_a}(\mathbf{Z}_i^*, \mathbf{Z}_i^-)}{Q_{q_a}(\mathrm{d}^r)}, 1\Big), \tag{11}$$

where d^r is the vector of correction divergences and q_a is the activation quantile. Nodes whose teacher logits differ substantially from the baseline before correction receive higher distillation emphasis, while unchanged nodes contribute minimally, focusing computational resources on the most critical regions.

Require: Trained GNN f_θ; cached original logits $\mathbf{Z}$; post-unlearning graph $\mathcal{G}^-$; retained training labels; unlearning request Δ (with node targets $\Delta\mathcal{V}$ for node unlearning); hyperparameters $\tau, \lambda, \mu, K, \lambda_{\text{kd}}, T, E, E_w$

Ensure: Unlearned model $f_{\theta'}$

Stage 1: Structural Perturbation Response Sensing

Load cached original logits $\mathbf{Z}$
$\mathbf{Z}^- \leftarrow f_\theta(\mathcal{G}^-)$
Compute response field $\mathbf{r}$ using Eq. (4)
if node unlearning **then**
  for all $i \in \Delta\mathcal{V}$ **do**
    $r_i \leftarrow 1$
  end for
end if

Stage 2: Field-Guided Residual Logit Correction

$\boldsymbol{\delta}^{(0)} \leftarrow \mathbf{0}$
Compute edge gates e_{ij}, prior weights w_i^p, and margin targets $\mathbf{R}_i$
for $t = 1$ to K **do**
  Update $\boldsymbol{\delta}^{(t)}$ using Eq. (7)
end for
$\mathbf{Z}^* \leftarrow \mathbf{Z}^- + \boldsymbol{\delta}^{(K)}$

Stage 3: Response-Adaptive Distillation

Compute adaptive weights $\mathbf{w}$ using Eq. (11)
Initialize student parameters $\theta' \leftarrow \theta$
for epoch $= 1$ to E **do**
  Compute $\mathcal{L}$ using Eq. (12) ▷ warm up for first E_w epochs
  Update θ' via gradient descent on $\mathcal{L}$
end for
return $f_{\theta'}$

Algorithm 1 Pseudocode for SURGE, which uses a structural response field to guide residual logit correction and response-adaptive distillation for graph unlearning

4.4.2 Student Training Objective

We initialize the student model $f_{\theta'}$ from the original trained model f_θ and optimize on the unlearned graph $\mathcal{G}^-$ with:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_{\text{kd}}\, \mathcal{L}_{\text{KD}}^{w}, \tag{12}$$

where the cross-entropy loss on retained training nodes maintains classification accuracy:

$$\mathcal{L}_{\text{CE}} = \frac{1}{|\mathcal{V}_L|} \sum_{i \in \mathcal{V}_L} \ell_{\text{CE}}(f_{\theta'}(\mathcal{G}^-)_i,\, y_i), \tag{13}$$

and the weighted knowledge distillation loss transfers the corrected knowledge:

$$\mathcal{L}_{\mathrm{KD}}^{w} = \frac{\sum_i w_i \cdot T^2 \cdot D_{\mathrm{KL}}\left(\sigma_T(\mathbf{Z}_i^*) \,\middle\|\, \sigma_T(\mathbf{Z}_{s,i})\right)}{\sum_i w_i}, \tag{14}$$

with student logits $\mathbf{Z}_s = f_{\theta'}(\mathcal{G}^-)$ and distillation temperature T. The T^2 scaling follows the standard convention (Hinton et al., 2015). To ensure stable optimization, we employ a warmup strategy where only $\mathcal{L}_{\mathrm{CE}}$ is active for the first E_w epochs, allowing the student to adapt to the post-unlearning graph structure before incorporating the teacher's guidance.

4.5 Algorithm and Complexity Analysis

The pseudocode for SURGE is presented in Algorithm 1. With cached original logits, the Sense stage requires one GNN forward pass on $\mathcal{G}^-$ with complexity $O(|\mathcal{E}^-| \cdot d)$, where d is the feature dimension; if the cache is unavailable, an optional read-only forward pass on $\mathcal{G}$ is needed to construct $\mathbf{Z}$. The Repair stage performs K iterations of sparse message passing, costing $O(K \cdot |\mathcal{E}^-| \cdot c)$ with c classes ($K = 20$ in all experiments). The Distill stage requires E epochs of standard GNN training on $\mathcal{G}^-$. The Sense and Repair stages are entirely gradient-free, involving no backpropagation, and the total cost is dominated by the lightweight Distill stage, which requires significantly fewer epochs than full retraining.

5 Experiments

In this section, we carry out extensive experiments to assess the effectiveness of SURGE for graph unlearning. The experiments are organized to answer the following research questions:

- **Q1:** How well does SURGE preserve predictive utility across different deletion requests?
- **Q2:** Does SURGE effectively forget the requested information?
- **Q3:** How sensitive is SURGE to key hyperparameters?
- **Q4:** How do SURGE's component choices affect performance?

5.1 Experimental Setup

5.1.1 Datasets

We conduct experiments on eight widely used transductive node classification benchmarks spanning citation networks, co-purchase graphs, co-authorship graphs, and a large-scale academic graph. The detailed statistics are summarized in Table 2.

Cora (McCallum et al., 2000; Yang et al., 2016), **CiteSeer** (Giles et al., 1998; Yang et al., 2016), and **PubMed** (Sen et al., 2008; Yang et al., 2016) are citation networks where nodes represent publications with bag-of-words features and labels denote research topics. They serve as canonical semi-supervised node classification benchmarks.

Table 2 Statistics of the benchmark datasets used for node classification and graph unlearning evaluation

Dataset	#Nodes	#Features	#Edges	#Classes
Cora	2,708	1,433	5,429	7
CiteSeer	3,327	3,703	4,732	6
PubMed	19,717	500	44,338	3
Amazon Photo	7,487	745	119,043	8
Amazon Computers	13,381	767	245,778	10
Coauthor CS	18,333	6,805	81,894	15
Coauthor Physics	34,493	8,415	247,962	5
ogbn-arxiv	169,343	128	1,166,243	40

Amazon Photo and **Amazon Computers** (McAuley et al., 2015; Shchur et al., 2018) are co-purchase graphs in which nodes represent products and edges indicate frequent co-purchasing. These two graphs are notably denser than citation networks, making them effective testbeds for evaluating how unlearning influence propagates through dense message-passing paths.

Coauthor CS and **Coauthor Physics** (Shchur et al., 2018; Wang et al., 2020) are co-authorship graphs extracted from the Microsoft Academic Graph, where nodes are authors and edges denote co-authorship relations. These datasets provide moderate- to large-scale evaluation scenarios with relatively high homophily.

ogbn-arxiv (Hu et al., 2020) is a large-scale citation network from the Open Graph Benchmark, comprising over 169K nodes and 1.1M edges. We include it in our ablation study (Table 18) to examine scalability on a substantially larger graph.

Following the protocol adopted in recent graph unlearning studies, we randomly split the nodes of each dataset into 80% for training and 20% for testing, and generate unlearning requests from the training portion.

5.1.2 Backbone GNNs

To evaluate the generalizability of SURGE across different architectures, we employ several representative GNN models as backbone classifiers.

GCN (Kipf & Welling, 2017) performs spectral-inspired neighborhood aggregation through a first-order approximation of graph convolutions, establishing a foundational architecture for semi-supervised node classification.

GAT (Veličković et al., 2018) introduces a learnable attention mechanism that assigns adaptive importance weights to neighboring nodes during aggregation, capturing heterogeneous neighborhood contributions.

GIN (Xu et al., 2019) adopts a sum-based aggregation scheme with multilayer perceptrons to achieve maximal expressiveness among message-passing GNNs.

GraphSAGE (Hamilton et al., 2017) is an inductive GNN framework that generates node embeddings by sampling and aggregating features from local neighborhoods.

We use 2-layer GCN and GAT as default backbones throughout the main experiments. GIN and GraphSAGE are used in the edge attack evaluation (Fig. 3) to verify backbone-agnostic robustness.

5.1.3 Baselines

We treat training from scratch on the remaining graph (Retrain) as the oracle reference. We compare SURGE with mainstream baselines from three representative paradigms:

Partition-based methods: GraphEraser (Chen et al., 2022) partitions the graph into disjoint shards and retrains only affected submodels (we evaluate balanced k-means and balanced label propagation variants); GUIDE (Wang et al., 2023) extends this paradigm with improved partitioning and similarity-based aggregation.

Influence function methods: GIF (Wu et al., 2023) derives closed-form parameter updates using graph-aware influence functions; ETR (Yang et al., 2025) achieves training-free unlearning through parameter erasure and rectification; IDEA (Dong et al., 2024) performs certified unlearning via Hessian-based parameter-space editing with noise injection.

Learning-based methods: GNNDelete (Cheng et al., 2023) introduces layerwise deletion operators; MEGU (Li et al., 2024) jointly trains predictive and unlearning modules through mutual evolution.

To reduce the effect of randomness and enable fair comparison, all methods compared within the same setting use identical train/test splits, unlearning requests, retraining references, and random seeds. Unless otherwise stated, the main 10% comparison tables are averaged over ten seeds; focused diagnostic studies report their own seed counts in the corresponding sections. When ETR, MEGU, and IDEA are included, they are run from their official repositories under this matched-request protocol. Baseline hyperparameters follow the authors' recommended settings unless a method provides dataset-specific defaults; SURGE uses the validation protocol described below. This protocol compares method behavior under identical unlearning requests, rather than exhaustively re-optimizing every baseline over a larger search space.

5.1.4 Unlearning Tasks

We consider three types of GU requests: **node unlearning** (randomly select training nodes as unlearning targets, remove all their incident edges, and exclude them from the training set), **edge unlearning** (uniformly sample edges and delete them), and **feature unlearning** (randomly select training nodes and zero out their input features). The unlearning ratio is set to 10% for all tasks unless otherwise stated.

5.1.5 Hyperparameter Settings

SURGE involves two groups of hyperparameters: (i) fixed hyperparameters that remain constant across all datasets and backbones, and (ii) tuned hyperparameters that are adjusted per dataset via grid search on a small validation split.

Fixed hyperparameters. Temperature parameters: $T_s = T_a = T = 2.0$. Correction parameters: label supervision weight $\mu = 1.0$, margin strength $m = 3.0$, prior power $\kappa = 1.0$, prior floor $\alpha = 0.05$, numerical stability constant $\varepsilon = 10^{-12}$, solver iterations $K = 20$. Response field normalization quantile: $\tau = 0.99$. Distillation parameters: warmup epochs $E_w = 5$, label smoothing $\epsilon_s = 0.05$, gradient clipping threshold $\gamma_{\text{clip}} = 2.0$, weight decay $\lambda_{\text{wd}} = 0$.

Tuned hyperparameters. Three hyperparameters are tuned per dataset: correction smoothness $\lambda \in \{0.01, 0.05, 0.1, 0.5\}$, activation quantile $q_a \in \{0.90, 0.95, 0.99\}$, and KD

weight $\lambda_{\text{kd}} \in \{0.01, 0.05, 0.1\}$. The distillation learning rate is set to 0.2× the base learning rate used during original model training, and the total distillation epochs are fixed at $E = 35$. Default values ($\lambda = 0.05$, $q_a = 0.99$, $\lambda_{\text{kd}} = 0.05$) are used unless otherwise specified. The sensitivity analysis in Sect. 5.5 demonstrates that SURGE is robust across a broad range of these values.

The complete notation of all hyperparameters is summarized in Table 3.

5.1.6 Evaluation Metrics

The utility tables report micro-F1 for single-label multiclass node classification. In this setting, micro-F1 is numerically equal to classification accuracy; it is distinct from class-macro F1. The aggregate utility values in Table 12 average the corresponding cell-level micro-F1 scores across 18 dataset–backbone–task settings. Retraining faithfulness is measured separately through JSD and probability MSE between the unlearned and retrained output distributions, for which lower values indicate closer agreement.

5.1.7 Experiment Environment

Experiments are conducted with an Intel Core i9-14900K CPU and a single NVIDIA GeForce RTX 5090 with 32 GB GPU memory. The operating system is Ubuntu 24.04.3 LTS. Software versions: Python 3.10.18, PyTorch 2.8.0, and CUDA 13.0.

5.2 Model Utility Preservation Across Unlearning Tasks

For **Q1**, we examine whether an unlearned model can retain predictive accuracy on the retained graph.

Table 3 SURGE hyperparameters. Fixed values are used across experiments; tuned parameters are selected from the listed grids

Symbol	Description	Value
ε	Numerical stability constant	10^{-12}
α	Prior floor in w_i^p	0.05
ϵ_s	Label smoothing coefficient	0.05
T_s, T_a, T	Temperatures (sensing, activation, KD)	2.0
μ	Label supervision weight	1.0
m	Margin strength	3.0
κ	Prior power exponent	1.0
τ	Response field normalization quantile	0.99
K	Solver iterations	20
E_w	Warmup epochs	5
E	Total distillation epochs	35
γ_{clip}	Gradient clipping	2.0
λ	Correction smoothness (tuned)	$\{0.01, \ldots, 0.5\}$
q_a	Activation quantile (tuned)	$\{0.90, 0.95, 0.99\}$
λ_{kd}	KD weight (tuned)	$\{0.01, 0.05, 0.1\}$

5.2.1 Node Unlearning

Table 4 reports F1 scores for node classification after node unlearning under two backbones (GCN and GAT) across seven benchmarks.

Overall, SURGE delivers the strongest utility profile among the unlearning methods in Table 4: it is best or second-best in most settings and remains consistently close to the retraining reference. ETR and IDEA are competitive on several citation-graph settings, while SURGE is especially stable on denser co-purchase and co-authorship graphs. This pattern supports the value of the structural perturbation response field, which estimates nodes whose predictions are most sensitive to the graph modification. The field then guides residual logit correction and response-weighted distillation in a backbone-agnostic manner, preserving utility without relying on fixed-neighborhood heuristics.

ETR demonstrates strong performance through its dedicated rectification stage, while MEGU preserves utility by explicitly optimizing the predictive objective during unlearning. IDEA pursues a different goal—certified unlearning—by approximating the retraining optimum via Hessian-based parameter correction and injecting calibrated Gaussian noise for (ϵ, δ) certification. While IDEA achieves competitive results on several datasets, its utility can degrade in scenarios where approximation error and certification noise become pronounced. In contrast, SURGE synthesizes an output-space teacher via field-guided residual logit correction on the unlearned graph and only distills where corrections are needed, leading to more stable utility across diverse settings.

In certain cases, SURGE slightly exceeds the retraining reference, particularly on co-purchase graphs with GCN. This phenomenon has also been observed in prior work (Li et al., 2024) and may arise because removing unlearning entities can eliminate spurious correlations, while the warm-started, field-guided updates serve as implicit regularization on the retained graph. We therefore interpret these cases as retained-utility effects rather than evidence of better-than-oracle forgetting, and separately compare the unlearned model with the retrained reference using distributional faithfulness metrics below.

5.2.2 Edge Unlearning

Table 5 presents F1 scores under edge unlearning across all seven benchmark datasets with GCN and GAT backbones. SURGE achieves the highest utility in 11 out of 14 dataset–backbone configurations. The improvements are particularly notable on denser graphs such as Amazon Photo and Amazon Computers, where edge deletions trigger wider propagation effects through the message-passing mechanism. Under GAT, SURGE outperforms all baselines on six out of seven datasets, often matching or exceeding the Retrain oracle.

5.2.3 Feature Unlearning

Table 6 reports the corresponding results under feature unlearning, where a subset of node features are replaced with zeros. SURGE achieves the best F1 score in 10 out of 14 settings, with particularly consistent performance under the GAT backbone. On citation graphs under GCN, IDEA obtains notably higher accuracy on several datasets, likely because its Hessian-based parameter correction benefits from the relatively sparse structure and well-conditioned optimization landscape of these graphs. SURGE remains competitive in these

Table 4 Node unlearning utility on seven datasets with GCN/GAT backbones

Backbone	Strategy	Cora	CiteSeer	PubMed	Photo	Computer	CS	Physics
GCN	Retrain	87.51 ± 0.36	75.59 ± 0.48	86.74 ± 0.13	89.67 ± 0.24	84.73 ± 0.75	93.16 ± 0.23	94.18 ± 0.16
	GraphEraser	68.15 ± 0.98	62.54 ± 1.21	65.33 ± 1.02	48.21 ± 1.85	42.66 ± 1.57	69.43 ± 1.10	71.50 ± 0.95
	GIF	85.11 ± 0.45	73.50 ± 0.49	85.42 ± 0.36	87.58 ± 0.28	83.62 ± 0.31	91.85 ± 0.61	92.88 ± 0.43
	GUIDE	84.89 ± 0.47	73.12 ± 0.60	85.25 ± 0.38	87.15 ± 0.41	79.05 ± 0.81	90.11 ± 0.68	92.05 ± 0.55
	GNNDelete	86.95 ± 0.40	74.01 ± 0.55	85.04 ± 0.45	88.58 ± 0.25	84.10 ± 0.42	92.17 ± 0.58	93.06 ± 0.48
	MEGU	86.96 ± 0.63	73.15 ± 1.14	86.26 ± 0.46	89.29 ± 0.31	81.32 ± 1.04	92.51 ± 0.34	90.79 ± 1.55
	ETR	**88.01 ± 0.48**	75.63 ± 0.42	86.64 ± 0.19	89.36 ± 0.46	84.48 ± 0.83	93.29 ± 0.23	94.19 ± 0.23
	IDEA	86.61 ± 0.75	74.14 ± 0.38	85.41 ± 0.24	89.69 ± 0.23	83.55 ± 0.71	92.99 ± 0.22	93.85 ± 0.29
	SURGE	87.82 ± 0.68	**75.86 ± 0.39**	**86.73 ± 0.19**	**89.97 ± 0.20**	**85.05 ± 0.40**	**93.34 ± 0.17**	**94.35 ± 0.20**
GAT	Retrain	83.84 ± 1.59	74.56 ± 0.48	84.41 ± 0.37	91.48 ± 0.54	89.28 ± 0.52	90.45 ± 0.75	92.18 ± 0.77
	GraphEraser	71.33 ± 1.05	50.25 ± 1.88	70.50 ± 1.52	68.42 ± 1.66	65.30 ± 1.83	72.40 ± 1.50	85.21 ± 1.45
	GIF	80.15 ± 0.66	73.05 ± 0.75	83.55 ± 0.50	90.13 ± 0.55	85.75 ± 0.63	88.98 ± 0.82	91.10 ± 0.95
	GUIDE	80.01 ± 0.80	72.55 ± 0.88	83.10 ± 0.75	89.22 ± 0.68	84.68 ± 0.90	88.01 ± 0.99	90.18 ± 1.25
	GNNDelete	81.55 ± 0.72	73.58 ± 0.70	83.05 ± 0.60	89.88 ± 0.60	86.50 ± 0.55	89.50 ± 0.77	91.80 ± 0.85
	MEGU	83.12 ± 1.19	71.44 ± 2.06	84.21 ± 0.34	76.60 ± 3.70	86.81 ± 0.48	85.97 ± 2.08	86.61 ± 0.94
	ETR	83.82 ± 1.18	74.19 ± 0.72	84.38 ± 0.47	87.08 ± 4.88	88.95 ± 0.32	90.20 ± 0.64	92.30 ± 0.54
	IDEA	**84.63 ± 1.06**	73.81 ± 0.89	84.50 ± 1.27	84.03 ± 2.23	86.37 ± 1.27	89.64 ± 0.91	89.38 ± 2.04
	SURGE	84.43 ± 0.91	**74.47 ± 0.71**	**84.79 ± 0.35**	**91.61 ± 0.45**	**89.34 ± 0.33**	**90.51 ± 0.57**	**92.57 ± 0.51**

Values are F1 scores (%; higher is better); among unlearning methods, excluding the Retrain reference, the best and second-best results are in **bold** and underlined, respectively

Table 5 Edge unlearning utility on seven datasets with GCN/GAT backbones

Backbone	Strategy	Cora	CiteSeer	PubMed	Photo	Computer	CS	Physics
GCN	Retrain	87.49 ± 0.49	75.51 ± 0.51	86.75 ± 0.22	89.61 ± 0.38	84.66 ± 0.84	93.13 ± 0.20	94.22 ± 0.16
	GraphEraser	70.15 ± 2.85	61.11 ± 2.10	78.14 ± 0.95	77.52 ± 1.51	70.25 ± 1.88	88.05 ± 0.71	91.63 ± 0.55
	GIF	83.50 ± 0.71	72.43 ± 1.15	83.25 ± 0.60	85.10 ± 0.99	79.55 ± 1.02	92.01 ± 0.45	93.58 ± 0.31
	GUIDE	85.45 ± 0.60	74.33 ± 0.75	85.52 ± 0.45	87.50 ± 0.65	82.43 ± 0.91	92.25 ± 0.30	93.91 ± 0.25
	GNNDelete	86.85 ± 0.52	75.05 ± 0.60	86.15 ± 0.30	88.85 ± 0.45	84.02 ± 0.88	92.85 ± 0.25	94.02 ± 0.24
	MEGU	86.49 ± 0.38	74.02 ± 0.68	86.01 ± 0.51	87.12 ± 1.86	79.13 ± 1.13	92.63 ± 0.26	93.20 ± 0.66
	ETR	**88.03 ± 0.62**	75.50 ± 0.53	**86.64 ± 0.18**	89.00 ± 1.11	84.60 ± 0.52	93.26 ± 0.19	94.25 ± 0.18
	IDEA	85.15 ± 0.64	71.67 ± 0.86	84.93 ± 0.16	89.18 ± 0.32	83.56 ± 0.62	91.96 ± 0.20	93.78 ± 0.24
	SURGE	87.97 ± 0.47	**75.74 ± 0.41**	86.63 ± 0.21	**89.80 ± 0.23**	**84.91 ± 0.43**	**93.40 ± 0.15**	**94.26 ± 0.23**
GAT	Retrain	83.91 ± 0.68	74.26 ± 0.66	84.44 ± 0.27	91.48 ± 0.76	89.17 ± 0.41	90.67 ± 0.59	92.21 ± 0.83
	GraphEraser	55.25 ± 2.50	53.18 ± 2.33	70.61 ± 1.55	79.25 ± 2.11	75.18 ± 2.45	83.05 ± 1.02	88.09 ± 1.15
	GIF	81.95 ± 1.10	72.91 ± 1.35	83.04 ± 0.65	88.58 ± 1.05	87.03 ± 0.88	89.59 ± 0.82	91.26 ± 0.91
	GUIDE	80.15 ± 1.18	72.65 ± 0.90	82.81 ± 0.80	86.81 ± 1.79	85.01 ± 1.25	88.60 ± 0.92	90.81 ± 1.05
	GNNDelete	83.10 ± 0.85	73.81 ± 0.75	84.05 ± 0.40	90.95 ± 0.85	88.85 ± 0.55	90.32 ± 0.61	91.95 ± 0.88
	MEGU	83.38 ± 1.39	72.36 ± 1.71	84.12 ± 0.38	84.32 ± 4.36	79.74 ± 3.53	87.73 ± 1.62	88.64 ± 1.43
	ETR	83.75 ± 1.13	74.25 ± 0.74	84.50 ± 0.29	89.45 ± 1.63	89.30 ± 0.30	90.64 ± 0.40	92.36 ± 0.57
	IDEA	**84.72 ± 1.02**	73.48 ± 0.80	84.39 ± 0.50	86.46 ± 2.33	86.96 ± 1.14	89.84 ± 0.71	90.03 ± 1.96
	SURGE	84.17 ± 0.66	**74.28 ± 0.60**	**84.73 ± 0.35**	**91.68 ± 0.46**	**89.51 ± 0.45**	**90.67 ± 0.48**	**92.48 ± 0.51**

Values are F1 scores (%; higher is better); among unlearning methods, excluding the Retrain reference, the best results are in **bold**

Table 6 Feature unlearning utility on seven datasets with GCN/GAT backbones

Backbone	Strategy	Cora	CiteSeer	PubMed	Photo	Computer	CS	Physics
GCN	Retrain	87.62 ± 0.37	75.78 ± 0.31	86.65 ± 0.20	89.06 ± 0.61	84.34 ± 0.86	93.25 ± 0.22	94.04 ± 0.24
	GraphEraser	72.55 ± 1.98	63.18 ± 2.15	75.21 ± 1.88	78.15 ± 2.05	71.40 ± 2.53	85.67 ± 1.15	86.25 ± 1.30
	GIF	84.15 ± 0.85	72.55 ± 1.02	85.05 ± 0.75	86.20 ± 0.95	80.13 ± 1.25	91.50 ± 0.60	92.73 ± 0.55
	GUIDE	86.21 ± 0.65	74.10 ± 0.77	86.01 ± 0.45	87.35 ± 0.80	82.01 ± 1.00	92.33 ± 0.41	93.25 ± 0.42
	GNNDelete	87.10 ± 0.55	75.10 ± 0.61	86.25 ± 0.35	88.55 ± 0.70	83.50 ± 0.90	92.95 ± 0.28	93.81 ± 0.35
	MEGU	86.38 ± 0.93	72.93 ± 1.44	86.13 ± 0.74	89.05 ± 0.35	81.01 ± 1.11	92.47 ± 0.38	90.30 ± 1.78
	ETR	87.88 ± 0.55	75.51 ± 0.48	86.76 ± 0.24	88.82 ± 1.68	83.96 ± 0.60	93.18 ± 0.21	93.99 ± 0.26
	IDEA	**88.36 ± 0.39**	**76.95 ± 0.37**	**87.07 ± 0.19**	89.36 ± 0.27	83.08 ± 0.69	93.11 ± 0.17	93.89 ± 0.18
	SURGE	88.15 ± 0.47	75.60 ± 0.41	86.56 ± 0.21	**89.73 ± 0.30**	**84.73 ± 0.51**	**93.41 ± 0.14**	**94.06 ± 0.32**
GAT	Retrain	83.58 ± 1.28	74.34 ± 0.56	84.26 ± 0.40	91.73 ± 0.43	89.56 ± 0.46	90.60 ± 0.55	90.84 ± 1.44
	GraphEraser	68.42 ± 3.55	60.15 ± 3.10	72.55 ± 2.10	65.11 ± 7.50	67.24 ± 7.10	78.50 ± 6.01	81.25 ± 3.15
	GIF	78.10 ± 3.01	70.05 ± 2.50	81.50 ± 1.50	78.33 ± 4.10	75.88 ± 4.55	80.05 ± 5.80	85.50 ± 2.50
	GUIDE	80.75 ± 2.80	71.85 ± 1.75	83.01 ± 1.15	81.15 ± 3.25	80.22 ± 2.95	83.15 ± 4.25	88.15 ± 2.05
	GNNDelete	82.85 ± 1.85	73.55 ± 0.95	83.95 ± 0.65	90.88 ± 0.95	88.95 ± 0.75	89.95 ± 0.88	90.21 ± 1.55
	MEGU	80.31 ± 2.65	70.99 ± 2.71	83.02 ± 1.05	67.34 ± 8.11	69.53 ± 6.49	81.19 ± 5.49	83.43 ± 2.74
	ETR	83.28 ± 2.00	74.26 ± 0.70	**84.52 ± 0.35**	87.78 ± 3.74	89.33 ± 0.31	90.59 ± 0.55	91.66 ± 1.07
	IDEA	81.73 ± 2.87	72.57 ± 1.28	83.93 ± 0.87	83.37 ± 2.24	86.62 ± 1.29	89.19 ± 1.31	89.67 ± 1.84
	SURGE	**84.17 ± 1.09**	**74.32 ± 0.66**	84.49 ± 0.36	**91.82 ± 0.53**	**89.54 ± 0.28**	**90.71 ± 0.44**	**91.89 ± 0.73**

Values are F1 scores (%; higher is better); among unlearning methods, excluding the Retrain reference, the best results are in **bold**

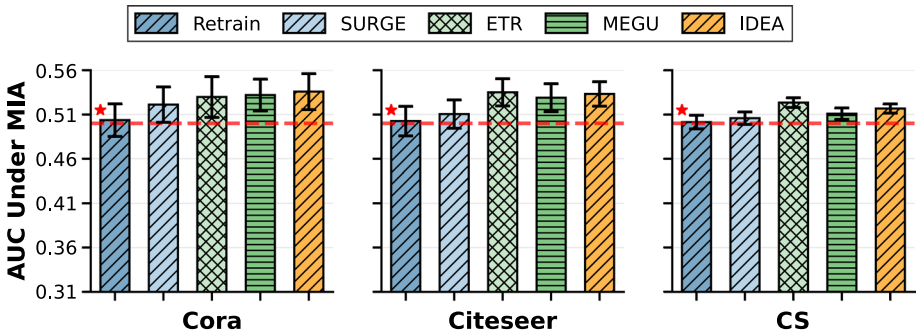


Fig. 2 Loss-based membership inference diagnostic under the GAT backbone. AUC values closer to the random-guess reference of 0.5 indicate lower membership distinguishability; the comparison measures alignment with Retrain under this attack

Table 7 Loss-based MIA evaluation for SURGE

Type	Group	D–B–T cells	Runs	Base	SURGE	Retr	Abs. gap
Overall	All	18	180	0.5201	0.5228	0.5193	0.0083
Dataset	Cora	6	60	0.5342	0.5340	0.5195	0.0145
	PubMed	6	60	0.5169	0.5179	0.5213	0.0053
	Amazon Comp	6	60	0.5090	0.5166	0.5173	0.0052
Task	Node	6	60	0.5084	0.5089	0.5017	0.0089
	Edge	6	60	0.5434	0.5463	0.5466	0.0102
	Feature	6	60	0.5084	0.5132	0.5096	0.0059
Backbone	GCN	9	90	0.5248	0.5277	0.5257	0.0076
	GAT	9	90	0.5153	0.5179	0.5130	0.0091

Each dataset–backbone–task (D–B–T) cell is averaged over ten seeds; marginal blocks summarize the same 18 cells. AUC values closer to 0.5 indicate lower attack distinguishability

cases and is strongest on denser co-purchase and co-authorship graphs under both backbones, where feature perturbations propagate through richer message-passing paths.

5.3 Forgetting Effectiveness Assessment

To answer **Q2** regarding whether SURGE approaches the behavior of retraining after removal, we evaluate faithfulness and forgetting behavior using complementary distributional and loss-based diagnostics.

5.3.1 Loss-Based Membership-Inference Diagnostic

We use a loss-based membership inference attack (MIA) to compare unlearned and retrained models under a single attack model (Carlini et al., 2022; Shokri et al., 2017).

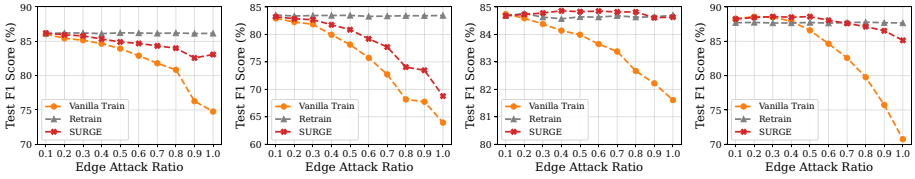
Figure 2 reports the GAT diagnostic on Cora, CiteSeer, and CS. The figure complements the retraining-faithfulness metrics below by showing membership distinguishability under the same loss-based attack.

The loss-based MIA AUC evaluation in Table 7 covers Cora, PubMed, and Amazon Computers with GCN/GAT backbones, node/edge/feature unlearning, and ten seeds. A data-

Table 8 Loss-based MIA evaluation across methods on the same 18 dataset–backbone–task cells

Method	Seed runs	Abs. AUC gap to Retrain
SURGE	180	0.0083
ETR	180	0.0113
MEGU	180	0.0135
IDEA	180	0.0083

Values are mean absolute AUC gaps from retraining over 180 seed runs per method; lower values indicate closer behavior under this diagnostic



(a) GCN Backbone (b) GAT Backbone (c) GIN Backbone (d) GraphSAGE Backbone

Fig. 3 Adversarial-edge unlearning on PubMed. Noisy cross-class edges are deleted as unlearning targets, and F1 is reported for four GNN backbones across attack ratios

set–backbone–task (D–B–T) cell denotes one setting averaged over ten seeds; the dataset, task, and backbone blocks are marginal summaries over the same 18 cells. The macro AUC is 0.5228 for SURGE and 0.5193 for Retrain, with a mean absolute gap of 0.0083. Table 8 reports the same gap for the external methods. The small gaps indicate close agreement with retraining under this loss-based attack.

5.3.2 Adversarial Edge Robustness

To evaluate unlearning capability under adversarial conditions, we adopt the edge attack protocol where noisy edges connecting nodes from different classes are injected into the training graph. These adversarial edges serve as unlearning targets, and we assess whether the unlearned model can mitigate their harmful effects. We introduce **vanilla train** as a reference baseline, which represents a model trained directly on the contaminated graph without any unlearning mechanism. Figure 3 illustrates the performance comparison on the PubMed dataset across four backbone architectures under varying edge attack ratios from 0.1 to 1.0.

As the attack ratio increases, the vanilla train model exhibits severe performance degradation, with F1 scores dropping dramatically across all backbones. In contrast, SURGE substantially recovers utility after deleting the adversarial edges and often approaches the retrained baseline on PubMed. The GAT backbone shows particular sensitivity to edge attacks due to its attention-based aggregation mechanism, yet SURGE still mitigates a large fraction of the adversarial edge influence.

The Cora evaluation in Table 9 uses attack ratios from 0.1 to 1.0, four backbones, and three seeds. For each backbone, the Δ_B row reports SURGE F1 minus contaminated Base F1, and the Δ_R row reports SURGE F1 minus clean Retrain F1 at the corresponding attack ratio. SURGE improves over contaminated vanilla training in 19 out of 20 backbone–ratio cells, and the average improvement over vanilla is positive for all four backbones. At the

Table 9 Cora adversarial-edge evaluation. Δ_B measures SURGE's F1 gain over contaminated Base training, and Δ_R measures SURGE's F1 difference relative to clean Retrain; positive Δ_B indicates mitigation of adversarial-edge contamination

Backbone	Metric	0.1	0.3	0.5	0.7	1.0	Avg. Δ_B
GCN	Δ_B	+0.0018	+0.0209	+0.0474	+0.0554	+0.0855	+0.0422
	Δ_R	+0.0018	−0.0018	−0.0301	−0.0621	−0.1230	–
GAT	Δ_B	+0.0178	+0.0264	+0.0541	+0.1150	+0.0855	+0.0598
	Δ_R	+0.0031	−0.0307	−0.0578	−0.1205	−0.2429	–
GIN	Δ_B	−0.0012	+0.0025	+0.0055	+0.0080	+0.0351	+0.0100
	Δ_R	−0.0006	+0.0043	+0.0018	−0.0037	+0.0105	–
SAGE	Δ_B	+0.0018	+0.0018	+0.0086	+0.0006	+0.0123	+0.0050
	Δ_R	−0.0062	−0.0068	−0.0123	−0.0123	−0.0135	–

Table 10 Retraining faithfulness for node unlearning measured by Jensen–Shannon (JS) divergence on the retain set

Backbone	Method	PubMed	CiteSeer	Cora	Photo	Computer	CS	Physics
GCN	MEGU	1.84e−3	9.59e−3	2.93e−3	4.28e−3	1.48e−2	7.39e−3	4.83e−3
	ETR	**2.38e−4**	**2.16e−3**	**2.35e−3**	1.04e−2	6.35e−3	**1.88e−3**	**8.31e−4**
	IDEA	2.98e−3	1.66e−2	1.32e−2	5.98e−3	6.37e−3	1.90e−2	5.95e−3
	SURGE	1.33e−3	4.65e−3	3.14e−3	**4.02e−3**	**5.29e−3**	6.49e−3	1.41e−3
GAT	MEGU	3.06e−3	9.24e−3	1.87e−2	1.17e−1	1.30e−1	9.60e−2	1.30e−2
	ETR	**1.72e−3**	4.50e−3	1.45e−2	4.98e−2	**9.24e−3**	1.79e−2	**3.15e−3**
	IDEA	2.85e−3	5.86e−3	1.06e−2	4.77e−2	2.51e−2	1.72e−2	1.15e−2
	SURGE	2.46e−3	**4.15e−3**	**7.19e−3**	**1.52e−2**	1.05e−2	**1.57e−2**	6.34e−3

Lower divergence indicates closer alignment with retraining; best results per backbone are in **bold**

highest attack ratios, especially for GAT, a gap to clean-graph retraining remains; together with the PubMed trend in Fig. 3, the evidence supports adversarial-edge mitigation on the tested datasets.

5.3.3 Activation Distribution Alignment

To evaluate the quality of unlearning, we measure the Jensen–Shannon (JS) divergence (Lin, 1991) between the output distributions of the unlearned model and the retrained model on the retain set. A well-executed unlearning method should produce predictions on retain nodes that closely match those of a model retrained from scratch without the forget data.

Table 10 presents comprehensive results across seven datasets with GCN and GAT backbones. Under GCN, ETR achieves the lowest JSD on several citation graphs (PubMed, CiteSeer, Cora), which is consistent with its training-free parameter rectification design that directly targets the parameter-space gap. Under GAT, where the attention mechanism makes aggregation more nonlinear, SURGE achieves the lowest JSD on four out of seven datasets, suggesting that output-space correction provides competitive alignment when the model's aggregation is more complex. Overall, SURGE attains the lowest JSD in six of the 14 dataset–backbone configurations and remains competitive in the other settings.

Table 11 Retraining faithfulness for node unlearning measured by probability MSE to the retrained model on the retain set

Backbone	Method	PubMed	CiteSeer	Cora	Photo	Computer	CS	Physics
GCN	MEGU	1.21e−3	2.34e−3	5.37e−4	**5.25e−4**	2.26e−3	4.20e−4	1.89e−3
	ETR	**1.93e−4**	**5.11e−4**	**3.69e−4**	1.78e−3	9.97e−4	**8.98e−5**	**2.25e−4**
	IDEA	4.63e−3	4.61e−3	2.70e−3	9.22e−4	8.44e−4	1.15e−3	2.15e−3
	SURGE	7.52e−4	1.18e−3	5.11e−4	6.54e−4	**5.24e−4**	4.29e−4	3.24e−4
GAT	MEGU	2.48e−3	2.65e−3	3.60e−3	2.60e−2	2.83e−2	1.01e−2	3.60e−3
	ETR	**1.52e−3**	1.07e−3	3.08e−3	1.00e−2	1.44e−3	1.80e−3	**1.18e−3**
	IDEA	3.03e−3	1.61e−3	2.25e−3	1.02e−2	4.28e−3	1.68e−3	2.21e−3
	SURGE	1.85e−3	**9.04e−4**	**1.51e−3**	**2.76e−3**	**1.39e−3**	**1.42e−3**	2.36e−3

Lower MSE indicates closer alignment with retraining; best results per backbone are in **bold**

Table 12 Mean utility and retraining faithfulness for node, edge, and feature unlearning over 18 dataset–backbone–task cells, each averaged over ten seeds

Higher micro-F1 and lower JSD/MSE are better

Method	Utility micro-F1 ↑	Test JSD ↓	Test MSE ↓	Retain MSE ↓	Forget MSE ↓
SURGE	**0.8632**	**0.0243**	**0.0088**	**0.0089**	**0.0100**
ETR	0.8605	0.0490	0.0127	0.0128	0.0138
MEGU	0.8300	0.0542	0.0146	0.0143	0.0162
IDEA	0.8509	0.0428	0.0119	0.0122	0.0125

5.3.4 Probability MSE to Retrained Model

In addition to the JS divergence, Table 11 provides a complementary view of the gap to the retrained model using the mean squared error (MSE) between predicted class probabilities of the unlearned model and the Retrain oracle. Results are computed on the retain set. SURGE achieves the lowest MSE in six out of 14 settings. Under GCN, ETR attains the lowest MSE on PubMed, CiteSeer, Cora, CS, and Physics, MEGU is lowest on Photo, and SURGE is lowest on Computers. Under GAT, SURGE attains the lowest MSE on five out of seven datasets, indicating that its output-space correction remains competitive when the backbone involves complex aggregation mechanisms.

5.3.5 Node, Edge, and Feature Faithfulness Comparison

To examine whether retraining-faithfulness trends extend beyond node unlearning, we evaluate Cora, PubMed, and Amazon Computers with GCN and GAT backbones, covering node, edge, and feature unlearning at a 10% deletion ratio over ten seeds. The comparison evaluates SURGE with ETR, MEGU, and IDEA under the same data splits, unlearning requests, seeds, and retraining references. All external methods are evaluated through the same retrained-oracle metrics after being trained in their respective official repositories.

Table 12 aggregates the 18 dataset–backbone–task cells, each averaged over ten seeds. Utility is reported as the mean of the cell-level micro-F1 scores, while JSD and MSE summarize agreement with retraining. The utility values, computed directly from the Cora, PubMed, and Amazon Computers entries in Tables 4, 5, 6, are 0.8632 for SURGE, 0.8605

Table 13 Paired win/tie/loss counts for SURGE against each baseline across the same 18 dataset–backbone–task cells

Metric	Baseline	SURGE better	Tie	SURGE worse
Utility micro-F1 ↑	ETR	15	3	0
	MEGU	18	0	0
	IDEA	16	1	1
Test JSD ↓	ETR	9	3	6
	MEGU	13	2	3
	IDEA	11	3	4
Test MSE ↓	ETR	10	2	6
	MEGU	12	2	4
	IDEA	11	1	6

Ties are determined by paired 95% confidence intervals

Table 14 Deletion-ratio robustness of SURGE by dataset and backbone

Dataset	Backbone	F1@5%	F1@10%	F1@20%	ΔF1@20%	JSD@5%	JSD@20%
Amazon Comp.	GAT	0.8609	0.8606	0.8592	−0.0318	0.0328	0.0343
	GCN	0.7882	0.7852	0.7845	−0.0607	0.0255	0.0284
Cora	GAT	0.8702	0.8682	0.8608	+0.0240	0.0275	0.0258
	GCN	0.8784	0.8776	0.8737	+0.0059	0.0040	0.0040
PubMed	GAT	0.8841	0.8836	0.8827	+0.0363	0.0330	0.0328
	GCN	0.8982	0.8975	0.8957	+0.0282	0.0188	0.0196
Macro	–	0.8633	0.8621	0.8594	+0.0003	0.0236	0.0242

Each row averages node, edge, and feature unlearning over three seeds; ΔF1@20% is measured relative to 20% retraining

for ETR, 0.8300 for MEGU, and 0.8509 for IDEA. SURGE also has the lowest mean divergence/MSE to the retrained oracle in this matched comparison. Table 13 reports paired win/tie/loss counts across the same 18 cells. The paired analysis shows strong utility results for SURGE, while faithfulness remains mixed in individual settings.

5.3.6 Deletion-Ratio Robustness

To evaluate sensitivity to request size, we sweep the unlearning ratio over 5%, 10%, and 20%. The sweep covers Cora, PubMed, and Amazon Computers with GCN/GAT backbones, node/edge/feature unlearning, and three seeds, comparing SURGE against the same-ratio retraining oracle. Table 14 reports the corresponding dataset–backbone summaries, with each row averaged over node/edge/feature unlearning and three seeds. From 5% to 20%, mean cell-level micro-F1 decreases moderately from 0.8633 to 0.8594, while Test JSD changes from 0.0236 to 0.0242. The trend indicates that SURGE remains stable under moderately larger single-request deletions, although the Amazon Computers rows show larger utility gaps to retraining than the citation-network rows. The evidence supports ratio robustness within the 5%–20% range.

5.3.7 Repeated Unlearning

We evaluate a short sequential-request setting to examine whether SURGE can be applied beyond a single one-shot deletion. The experiment uses node unlearning with two disjoint 5% training-node deletion requests, resulting in an approximately 10% cumulative dele-

Table 15 Two-step repeated node unlearning

Group	N	Seq. F1	1-shot F1	Ret. F1	Seq. JSD	1-shot JSD	Time ratio
All	18	0.8705	0.8621	0.8628	0.0198	0.0238	1.98
GCN	9	0.8638	0.8541	0.8650	0.0104	0.0165	1.93
GAT	9	0.8771	0.8701	0.8606	0.0291	0.0311	2.02
Cora	6	0.8736	0.8730	0.8625	0.0143	0.0148	1.90
PubMed	6	0.8893	0.8894	0.8556	0.0241	0.0253	2.00
Amazon Computers	6	0.8484	0.8239	0.8702	0.0209	0.0313	2.03

Seq. denotes refreshed-reference sequential SURGE, and 1-shot denotes a cumulative one-shot update; higher F1 indicates stronger utility, while lower JSD indicates closer retraining behavior

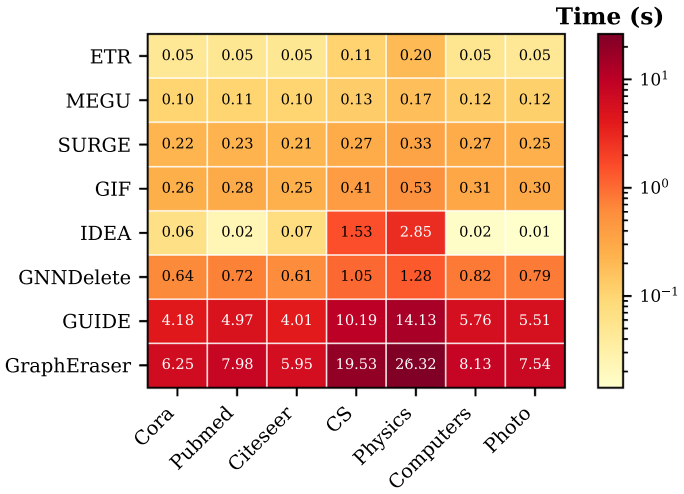


Fig. 4 Wall-clock unlearning time across seven datasets under a GCN backbone. Times are reported in seconds with logarithmic-scale coloring; lower is faster

tion. We compare three models on Cora, PubMed, and Amazon Computers with GCN/GAT backbones and three seeds: (i) a one-shot SURGE update on the cumulative deletion set, (ii) a sequential SURGE update that refreshes the reference after the first 5% request and then applies the second 5% request, and (iii) retraining from scratch on the cumulative post-deletion graph.

Table 15 shows that the refreshed sequential variant remains stable in this two-step setting. Averaged over all 18 dataset–backbone–seed runs, sequential SURGE obtains higher F1 than the one-shot cumulative update (0.8705 vs. 0.8621) and lower Test JSD to retraining (0.0198 vs. 0.0238), at roughly twice the one-shot unlearning time. The dataset rows show that the sequential update is especially helpful on Amazon Computers relative to the one-shot cumulative update. This setting illustrates short refreshed-reference deployment; longer request streams can use explicit reference-refresh policies, as discussed in Sect. 6.

Table 16 Cross-method runtime under a GAT backbone

Method	N	Mean (s) ↓	Std (s)	Range (s)
ETR	180	0.0570	0.0221	0.0359-–0.2083
SURGE	180	0.1522	0.0831	0.0779-–0.3153
MEGU	180	0.1894	0.1458	0.1237-–0.7934
IDEA	180	0.5458	1.0497	0.0100-–2.8962

Values are computed from matched dataset–task–seed cells where SURGE, ETR, MEGU, and IDEA all report runtime; lower is better

Table 17 Backbone-level SURGE runtime across matched deletion tasks

Backbone	N	SURGE (s) ↓	Retrain (s) ↓	Speedup ↑
GCN	90	0.0807	0.2336	2.89×
GAT	90	0.1019	0.3045	2.98×

Speedup is measured against same-setting retraining; lower time and higher speedup are better

5.4 Unlearning Efficiency

Figure 4 compares the wall-clock unlearning time of different methods under a GCN backbone across seven datasets. ETR exhibits consistently low runtime and achieves the lowest overall time across datasets, which aligns with its training-free parameter editing design. MEGU, while requiring extra optimization, is also fast in our setting. SURGE performs a lightweight teacher correction followed by a brief fine-tuning stage, resulting in moderate yet stable unlearning time. IDEA shows strong dataset dependence: it can be extremely fast on several datasets but suffers noticeable slowdowns on larger graphs such as CS and Physics. Partition-based retraining methods (GUIDE and GraphEraser) are the slowest, as they require retraining multiple shard models upon each request. Overall, SURGE offers a practical utility–faithfulness–runtime trade-off, with per-request runtimes well below shard retraining and full retraining.

The same practical pattern persists under an attention-based backbone. Table 16 reports 180 matched GAT timing cells across six datasets, three deletion tasks, and up to ten seeds, retaining only settings where all four methods expose a timing record. ETR remains the fastest method on average. SURGE is slower than ETR because it performs corrected-teacher construction and student distillation, but it remains faster than MEGU and IDEA on the matched GAT cells. The GAT comparison therefore reinforces the section's main message: SURGE is designed for a balanced utility–faithfulness–runtime trade-off rather than minimum latency in every setting.

Table 17 reports SURGE runtime relative to same-setting retraining on Cora, PubMed, and Amazon Computers over node, edge, and feature unlearning with ten seeds. GAT incurs a modest increase in wall-clock time compared with GCN (0.1019s vs. 0.0807s on average), while retaining a similar speedup over retraining (2.98× vs. 2.89×). This backbone-level view isolates the additional cost of attention-based aggregation within SURGE and complements the cross-method timing results above.

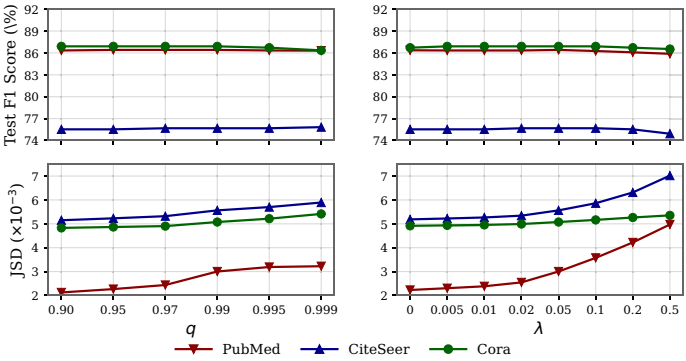


Fig. 5 Hyperparameter sensitivity of SURGE to activation quantile q_a and correction smoothness λ. Higher F1 and lower JSD indicate a stronger utility–faithfulness trade-off

Table 18 Module ablation on Amazon computers and ogbn-arxiv

Backbone	Module	Computers		ogbn-arxiv	
		JSD	Performance	JSD	Performance
GCN	w/o KD	0.0167	81.18 ± 1.75	0.0223	61.37 ± 0.69
	w/o *r*-field	0.0176	80.96 ± 1.68	0.0232	61.22 ± 0.73
	w/o teacher repair	0.0167	80.70 ± 1.85	0.0218	60.92 ± 0.74
	SURGE	0.0161	81.28 ± 1.80	0.0214	61.47 ± 0.64
GAT	w/o KD	0.0232	89.04 ± 0.34	0.0198	64.05 ± 0.39
	w/o *r*-field	0.0252	88.94 ± 0.35	0.0208	63.76 ± 0.37
	w/o teacher repair	0.0230	88.95 ± 0.32	0.0197	63.94 ± 0.38
	SURGE	0.0228	89.04 ± 0.40	0.0195	64.15 ± 0.39

F1 measures utility (higher is better), and JSD to the retrained model measures faithfulness (lower is better) for GCN/GAT backbones

5.5 Hyperparameter Sensitivity

SURGE is stable across a broad range of key hyperparameters. Figure 5 reports sensitivity to the teacher propagation strength λ and activation quantile q_a on representative datasets. Across a wide range of (q_a, λ), F1 score changes marginally, while the gap to the retrained model (JSD) varies smoothly. We observe that overly large q_a (overly sparse distillation) or overly large λ (excessive repair propagation) increases the gap to the retrained model, which aligns with the roles of these hyperparameters in SURGE.

5.6 Ablation Studies

We evaluate SURGE through core module ablations, paired component deltas, design-sensitivity checks, response-mechanism controls, and matched-budget response diagnostics. Together, these experiments separate the functional role of response-guided correction from the necessity of individual implementation choices.

Table 19 Component ablation for node and edge unlearning on Cora, PubMed, and Amazon Computers with GCN/GAT backbones and three seeds

Task	Variant	ΔF1	ΔJSD
Node	w/o KD	+0.0006	−0.0005
	uniform KD weights	+0.0002	−0.0007
	uniform response field	−0.0003	+0.0014
	w/o response label gate	−0.0001	+0.0006
	w/o edge gate	−0.0000	+0.0000
Edge	w/o KD	+0.0004	−0.0009
	uniform KD weights	+0.0003	−0.0011
	uniform response field	−0.0010	+0.0010
	w/o response label gate	−0.0003	+0.0002
	w/o edge gate	−0.0001	−0.0000

Values are paired mean deltas relative to full SURGE; lower Δ JSD indicates a closer match to retraining

Table 20 Node-unlearning design sensitivity for response scoring, target-node override, and distillation weighting

Variant	ΔF1	ΔJSD	F1 W/T/L	JSD W/T/L
Response-field KD weights	−0.0003	−0.0003	7/2/9	13/0/5
Uniform KD weights	+0.0002	−0.0007	8/2/8	16/0/2
No target-node override	+0.0000	−0.0000	0/18/0	3/10/5
Raw SKL response	+0.0002	−0.0005	8/6/4	15/0/3
JS-divergence response	−0.0003	−0.0002	2/10/6	14/0/4
Logit-L2 response	+0.0004	+0.0000	7/8/3	5/0/13
Probability-L2 response	−0.0004	−0.0003	5/7/6	15/0/3

Values are paired deltas relative to SURGE over 18 Cora/PubMed/Amazon Computers dataset–backbone–seed settings; lower ΔJSD indicates a closer match to retraining. W/T/L counts are descriptive; exact sign-flip tests use the six dataset–backbone cell means

Table 21 Response-mechanism controls on Cora, PubMed, and Amazon Computers with GCN/GAT backbones and node, edge, and feature unlearning

Control	ΔTest JSD	95% CI	Favorable cells	ΔTest MSE	ΔF1
Uniform response	+0.001071	[0.000491, 0.001619]	15/18	+0.000406	+0.000392
Shuffled response	+0.000459	[0.000148, 0.000769]	15/18	+0.000205	−0.000141

Each of the 18 dataset–backbone–task cells averages ten seeds. Positive JSD/MSE deltas mean that full SURGE is closer to retraining, and ΔF1 is full minus control. Confidence intervals use 10,000 bootstrap resamples of the 18 cell means

Table 18 first evaluates the core modules of SURGE by removing weighted KD, response-field gating, and teacher repair. Across Computers and ogbn-arxiv, the full model attains the lowest JSD and the strongest or tied F1 in the reported settings. Tables 19 and 20 refine this picture on the compact Cora/PubMed/Amazon Computers grid. Replacing the response field by a uniform field increases JSD, whereas no-KD and uniform-KD variants remain close to, and sometimes slightly better than, the full model. Removing the target-node override has essentially no effect at the reported precision. Exact sign-flip tests on the six dataset–backbone cell means give unadjusted JSD p-values of 0.0938/0.0313 for no-KD and 0.0625/0.0625 for uniform-KD in node/edge unlearning. After Holm correction, the reported compact-grid JSD contrasts do not distinguish these variants; adaptive KD weight-

Table 22 Response-field diagnostics versus fixed-hop neighborhoods on non-forget nodes

Signal	Top-5% agreement	Node coverage	Recall/coverage lift
Response ranking	0.0866	0.0500	1.73
1-hop mask	0.5422	0.5403	0.99
2-hop mask	0.7980	0.8593	0.92
3-hop mask	0.8984	0.9416	0.95

Recall/coverage lift above 1 means that the signal concentrates retraining-shifted nodes more selectively than its coverage alone would predict

Table 23 Matched-budget agreement with retraining-induced prediction shifts

Signal	NDCG at 5%
Response ranking	0.2537
Fixed-hop baseline	0.1957
PPR baseline	0.2232
Random ranking	0.2016

Each method ranks the same candidate nodes, and NDCG is evaluated at a 5% selection budget over the 18 dataset–backbone–task cells

ing, target-node override, and edge gating are therefore retained as default implementation choices rather than individually indispensable components.

Table 21 isolates the response signal from the remaining pipeline. Relative to a uniform field, full SURGE lowers cell-mean Test JSD by 0.001071 (95% CI [0.000491, 0.001619]) and Test MSE by 0.000406. Preserving the same response values but shuffling their node assignments reduces the JSD advantage to 0.000459 (95% CI [0.000148, 0.000769]), while the direction remains favorable in 15/18 cells. Mean F1 changes are small for both controls. Top-1 agreement is slightly lower for full SURGE, by 0.000523 relative to uniform response and 0.000416 relative to shuffled response. Together, these controls associate the response values and their node assignments with a modest distributional-faithfulness benefit, while leaving decision agreement nearly unchanged.

Tables 22 and 23 examine how the response ranking relates to retraining-induced prediction changes. Its top-5% agreement is 0.0866, corresponding to a 1.73 recall/coverage lift over random selection, while fixed-hop masks obtain high raw agreement largely by covering broad fractions of the graph. Under the matched 5% budget, response NDCG is 0.2537, compared with 0.1957 for fixed hop, 0.2232 for PPR, and 0.2016 for random ranking. For the paired cell-level analysis, the stronger topology baseline is defined as the larger of the two NDCG values obtained by the matched-budget fixed-hop and PPR rankings within each cell. The response ranking exceeds this per-cell baseline in 12/18 cells, with a mean paired advantage of +0.0156 and a bootstrap 95% CI of $[-0.0410, 0.0663]$. Its mean Spearman correlation with retraining-induced shifts is 0.0021. The near-zero mean correlation and the confidence interval containing zero indicate that the response score is an imperfect correction-routing signal rather than a stable ranking of retraining-induced shifts. Figure 6 provides qualitative examples of these response patterns for representative unlearning requests.

Taken together, the ablations support response-guided correction as the main design principle. The Distill stage has a distinct operational role: it transfers the corrected teacher logits from output space into deployable model parameters. The ablations position correction-magnitude weighting, symmetric-KL normalization, target-node override, and edge gating

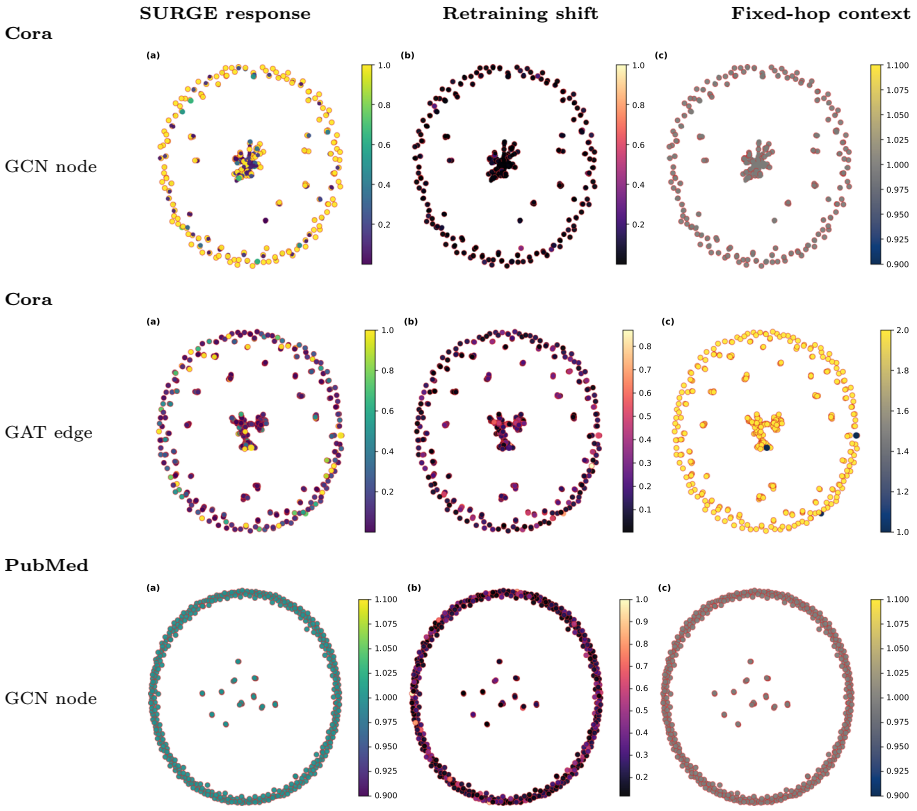


Fig. 6 Qualitative response maps for representative Cora and PubMed requests. Columns compare SURGE response scores, retraining-induced prediction changes, and fixed-hop context for the same request; rows specify the dataset, backbone, and unlearning task. Colorbars are normalized within each panel

as default design choices, while no-KD and uniform-KD remain viable alternatives in the compact evaluation.

6 Deployment Scope and Limitations

SURGE uses prediction shift as a request-conditioned correction-routing signal for graph unlearning. Proposition 1 provides only a one-way guarantee for adjacency perturbations: a nonzero output shift implies a changed normalized aggregation row, whereas a changed row can yield zero shift through cancellation. The response field should therefore be interpreted as an imperfect routing proxy rather than an exact localization or training-influence ranking. Its formal interpretation is most direct for node and edge deletion, where the request modifies graph connectivity. For feature unlearning, the same response definition remains useful as an output-space perturbation signal because erased features still propagate through message passing. Poor calibration or instability under small graph edits can make this signal

noisier; graph regularization and retained-label supervision provide additional constraints but do not turn it into an exact affected-region label.

Within the evaluated Cora/PubMed/Amazon Computers, GCN/GAT, and node/edge/feature grid, response controls improve distributional faithfulness on average, whereas F1 changes are small and the matched-budget comparison is inconclusive. We therefore interpret the response field as a context-dependent correction-routing signal.

SURGE assumes access to the trained model, the post-unlearning graph, the deletion request, retained supervision, and cached original logits. The cached logits can be precomputed after training and are smaller than raw graph features when the number of classes is modest, but they still represent model outputs and should be protected as part of the model state. The experiments include a 5%–20% single-request deletion-ratio sweep and a two-step refreshed-reference repeated-unlearning evaluation. For longer sequential deployments, SURGE can be paired with an explicit reference-refresh policy, such as refreshing logits after accepted requests or periodically anchoring to a retrained checkpoint.

The matched evaluation provides the broadest evidence for utility and retraining faithfulness, covering node, edge, and feature unlearning across datasets, backbones, and seeds. Because the MIA analysis uses a single loss-based attack and the Base-model AUC is close to random guessing, it serves as a retraining-alignment diagnostic rather than a broad privacy evaluation. Additional evidence covers runtime under both GCN and GAT through cross-method summaries and a SURGE-vs-retraining backbone view, and adversarial-edge unlearning on PubMed and Cora. Future work can extend the evaluation to stronger attack families and longer continual-unlearning streams.

7 Conclusion

We introduced SURGE, a practical three-stage framework for graph unlearning that follows a Sense–Repair–Distill pipeline. The key design is the structural perturbation response field, which measures the prediction sensitivity of the trained GNN to graph modifications and serves as a request-conditioned correction-routing proxy. This response field modulates a residual logit correction mechanism that constructs corrected teacher logits in the output space via a gradient-free graph-regularized solver. The Distill stage transfers those corrected logits into deployable model parameters; the compact ablations indicate that adaptive weighting is a default transfer policy rather than a uniformly necessary component. The Sense and Repair stages require no backpropagation.

Experiments on seven standard benchmarks across node, edge, and feature unlearning tasks with multiple GNN backbones demonstrate leading or competitive utility and forgetting effectiveness, while an ogbn-arxiv ablation supports scalability on a large graph. The matched node/edge/feature comparison shows the strongest mean utility and retraining faithfulness among SURGE, ETR, MEGU, and IDEA, with mixed per-setting faithfulness. Response-mechanism controls associate the response values and their request-conditioned assignments with a modest average JSD/MSE improvement. Together with the matched-budget analysis, these results position prediction-level structural response as a topology-adaptive correction signal rather than a general localization estimator.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/s10994-026-07176-x.

Author Contributions C.S. conceived the study, performed the experiments, analyzed the results, and wrote the original draft. M.W. reviewed and edited the manuscript and provided valuable suggestions for improvement. J.Z. supervised the research, provided guidance throughout the study, and reviewed the manuscript as the corresponding author. All authors have read and approved the final version of the manuscript.

Funding This work was funded by the Basic Research Program of Jiangsu under Grant No. BK20251747.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

Bourtoule, L., Chandrasekaran, V., Choquette-Choo, C. A., Jia, H., Travers, A., Zhang, B., & Papernot, N. (2021). *Machine unlearning. 2021 IEEE symposium on security and privacy* (pp. 141–159) arXiv:1912.03817.

California Department of Justice (2018). *California consumer privacy act (CCPA).* https://oag.ca.gov/privacy/ccpa (Accessed 2026-02-13)

Cao, Y., & Yang , J. (2015). Towards making systems forget with machine unlearning. *IEEE symposium on security and privacy* (pp. 463–480)

Carlini, N., Chien, S., Nasr, M., Song, S., Terzis, A., Tramèr, F. (2022). Membership inference attacks from first principles. *43rd IEEE symposium on security and privacy, SP 2022, san francisco, ca, usa, may 22–26, 2022* (pp. 1897–1914). IEEE. https://doi.org/10.1109/SP46214.2022.9833649

Chen H, Fan X, Liu H, Li Y, Zhao Z, Zhou F, Cao L (2026) Federated neural nonparametric point processes. Artificial Intelligence 351:104454

Chen, H., Fan, X., Wu, Z., & Cao, L. (2026b). FigBO: A generalized acquisition function framework with look-ahead capability for bayesian optimization. *Machine Learning, 115*(5), 110. https://doi.org/10.1007/s10994-026-07039-5. arXiv:2504.20307.

Chen, H., Liu, H., Wu, Z., Fan, X., & Cao, L. (2025). FedSI: Federated subnetwork inference for efficient uncertainty quantification. *IEEE Transactions on Neural Networks and Learning Systems, 36*(9), 16975–16989. https://doi.org/10.1109/TNNLS.2025.3562164. arXiv:2404.15657.

Chen, M., Zhang, Z., Wang, T., Backes, M., Humbert, M., Zhang, Y. (2022). Graph unlearning. *Proceedings of the 2022 ACM SIGSAC conference on computer and communications security* (pp. 499–513). (arXiv:2103.14991)

Cheng, J., Dasoulas, G., He, H., Agarwal, C., Zitnik, M. (2023). GNNDelete: A general strategy for unlearning in graph neural networks. *International conference on learning representations.* https://openreview.net/forum?id=X9yCkmT5Qrl (arXiv:2302.13406)

Chien, E., Pan, C., Milenkovic, O. (2022). *Certified graph unlearning.* https://neurips.cc/virtual/2022/63076 (NeurIPS 2022 Workshop: New Frontiers in Graph Learning)

Chundawat, V. S., Tarun, A. K., Mandal, M., & Kankanhalli, M. (2023). Can bad teaching induce forgetting? Unlearning in deep networks using an incompetent teacher. *Proceedings of the aaai conference on artificial intelligence* (Vol. 37, pp. 7210–7217)

Cong, W., & Mahdavi, M. (2023). Efficiently forgetting what you have learned in graph representation learning via projection. *Proceedings of the 26th international conference on artificial intelligence and statistics* (Vol. 206, pp. 6674–6703). PMLR. https://proceedings.mlr.press/v206/cong23a.html (arXiv:2302.08990)

Dai, E., Zhao, T., Zhu, H., Xu, J., Guo, Z., Liu, H., & Wang, S. (2024). A comprehensive survey on trustworthy graph neural networks: Privacy, robustness, fairness, and explainability. *Machine Intelligence Research, 21*(6), 1011–1061. https://doi.org/10.1007/s11633-024-1510-8. arXiv:2204.08570.

Dong, Y., Zhang, B., Lei, Z., Zou, N., & Li, J. (2024). IDEA: A flexible framework of certified unlearning for graph neural networks. *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 621–630) arXiv:2407.19398.

Fan, C., Liu, J., Zhang, Y., Wong, E., Wei, D., Liu, S. (2024). SalUn: Empowering machine unlearning via gradient-based weight saliency in both image classification and generation. *The twelfth international conference on learning representations, ICLR 2024, vienna, austria, may 7–11, 2024.* OpenReview.net. https://openreview.net/forum?id=gn0mIhQGNM

Foster, J., Schoepf, S., & Brintrup, A. (2024). Fast machine unlearning without retraining through selective synaptic dampening. *Proceedings of the AAAI conference on artificial intelligence* (pp. 12043–12051). https://doi.org/10.1609/aaai.v38i11.29092

GDPR-Info.eu (2016). *Art. 17 GDPR — right to erasure ("Right to Be Forgotten").* https://gdpr-info.eu/art-17-gdpr/ (Accessed 2026-02-13)

Giles, C. L., Bollacker, K. D., & Lawrence, S. (1998). CiteSeer: An automatic citation indexing system. *Proceedings of the third ACM conference on digital libraries* (pp. 89–98)

Ginart, A., Guan, M. Y., Valiant, G., & Zou, J. (2019). Making AI forget you: Data deletion in machine learning. *Advances in neural information processing systems* (pp. 3513–3526) arXiv:1907.05012.

Guo, C., Goldstein, T., Hannun, A., van der Maaten, L. (2020). Certified data removal from machine learning models. *Proceedings of the 37th international conference on machine learning* (Vol. 119, pp. 3832–3842). PMLR. https://proceedings.mlr.press/v119/guo20c.html

Gou, J., Yu, B., Maybank, S. J., & Tao, D. (2021). Knowledge distillation: A survey. *International Journal of Computer Vision, 129*(6), 1789–1819. https://doi.org/10.1007/s11263-021-01453-z

Hamilton, W. L., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. *Advances in neural information processing systems* (pp. 1024–1034) arXiv:1706.02216.

Hinton, G., Vinyals, O., Dean, J. (2015). *Distilling the knowledge in a neural network.* (arXiv:1503.02531)

Hu, W., Fey, M., Zitnik, M., Dong, Y., Ren, H., Liu, B., & Leskovec, J. (2020). Open graph benchmark: Datasets for machine learning on graphs. Advances in neural information processing systems arXiv:2005.00687.

Izzo, Z., Smart, M.A., Chaudhuri, K., Zou, J. (2021). Approximate data deletion from machine learning models. *Proceedings of the 24th international conference on artificial intelligence and statistics* (Vol. 130, pp. 2008–2016). PMLR. (arXiv:2002.10077)

Jia, J., Liu, J., Ram, P., Yao, Y., Liu, G., Liu, Y.. Liu, S. (2023). Model sparsity can simplify machine unlearning. *Advances in neural information processing systems 36: Annual conference on neural information processing systems 2023, neurips 2023, new orleans, la, usa, december 10–16, 2023* (pp. 51584–51605).

Kipf, T.N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. *International conference on learning representations.* https://openreview.net/forum?id=SJU4ayYgl (arXiv:1609.02907)

Kurmanji, M., Triantafillou, P., Hayes, J., Triantafillou, E. (2023). Towards unbounded machine unlearning. *Advances in neural information processing systems* (Vol. 36, pp. 1957–1987). (arXiv:2302.09880)

Lin, J. (1991). Divergence measures based on the Shannon entropy. *IEEE Transactions on Information Theory, 37*(1), 145–151. https://doi.org/10.1109/18.61115

Li, J., Shen, Y., Zhou, W., & Chen, H. (2026). SearchExpert: A GenAI-driven framework for reasoning-intensive multimedia information fusion through fine-tuning and reinforcement learning. *Information Fusion, 126*, Article 103665. https://doi.org/10.1016/j.inffus.2025.103665

Li, X., Zhao, Y., Wu, Z., Zhang, W., Li, R-H., Wang, G. (2024). Towards effective and general graph unlearning via mutual evolution. *Proceedings of the aaai conference on artificial intelligence* (Vol. 38, pp. 13682–13690). (arXiv:2401.11760)

McAuley, J., Targett, C., Shi, Q., van den Hengel, A. (2015). Image-based recommendations on styles and substitutes. *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval* (pp. 43–52).

McCallum, A., Nigam, K., Rennie, J., & Seymore, K. (2000). Automating the construction of internet portals with machine learning. *Information Retrieval, 3*(2), 127–163. https://doi.org/10.1023/A:1009953814988

Nguyen, T.T., Huynh, T.T., Ren, Z., Nguyen, P.L., Liew, A.W-C., Yin, H., Nguyen, Q.V.H. (2025). A survey of machine unlearning. *ACM Transactions on Intelligent Systems and Technology*, *16*(5), 108:1–108:46, https://doi.org/10.1145/3749987arXiv:2209.02299

Saad, Y. (2003). *Iterative methods for sparse linear systems* (2nd ed.). Philadelphia, PA: Society for Industrial and Applied Mathematics.

Said, A., Tran, N.N., Zhao, Y., Derr, T., Shabbir, M., Abbas, W., Koutsoukos, X. (2023). *A survey of graph unlearning.* https://doi.org/10.48550/arXiv.2310.02164arXiv:2310.02164

Sekhari, A., Acharya, J., Kamath, G., & Suresh, A. T. (2021). Remember what you want to forget: Algorithms for machine unlearning. *Advances in neural information processing systems* (Vol. 34, pp. 18075–18086)

Sen, P., Namata, G., Bilgic, M., Getoor, L., Gallagher, B., & Eliassi-Rad, T. (2008). Collective classification in network data. *AI Magazine, 29*(3), 93–106. https://doi.org/10.1609/aimag.v29i3.2157

Shchur, O., Mumme, M., Bojchevski, A., & Günnemann, S. (2018). Pitfalls of graph neural network evaluation arXiv:1811.05868.

Shokri, R., Stronati, M., Song, C., & Shmatikov, V. (2017). Membership inference attacks against machine learning models. *IEEE symposium on security and privacy* (pp. 3–18)

Tian, Y., Chen, H., Zhu, X., & Yu, X. (2026). FedKT: A nature-inspired bayesian federated learning framework for privacy-preserving and lightweight consumer devices. *IEEE Transactions on Consumer Electronics,1–1,*. https://doi.org/10.1109/TCE.2026.3667765

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y. (2018). Graph attention networks. *International conference on learning representations.* https://openreview.net/forum?id=rJXMpikCZarXiv:1710.10903

Wang, C-L., Huai, M., Wang, D. (2023). Inductive graph unlearning. *32nd USENIX security symposium* (pp. 3205–3222). https://www.usenix.org/conference/usenixsecurity23/presentation/wang-cheng-long

Wang, K., Shen, Z., Huang, C., Wu, C.-H., Dong, Y., & Kanakia, A. (2020). Microsoft academic graph: When experts are not enough. *Quantitative Science Studies, 1*(1), 396–413. https://doi.org/10.1162/qss_a_00021

Wang, R., Xu, D., Cascone, L., Wang, Y., Chen, H., Zheng, J., & Zhu, X. (2025). RAFT: Robust adversarial fusion transformer for multimodal sentiment analysis. *Array, 27*, Article 100445. https://doi.org/10.1016/j.array.2025.100445

Wu, J., Yang, Y., Qian, Y., Sui, Y., Wang, X., & He, X. (2023). GIF: A general graph unlearning strategy via influence function. *Proceedings of the ACM web conference 2023* (pp. 651–661) arXiv:2304.02835.

Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Yu, P. S. (2021). A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems, 32*(1), 4–24. https://doi.org/10.1109/TNNLS.2020.2978386

Xu, H., Zhu, T., Zhang, L., Zhou, W., Yu, P.S. (2024). Machine unlearning: A survey. *ACM Computing Surveys*, *56*(1), 9:1–9:36, https://doi.org/10.1145/3603620

Xu, K., Hu, W., Leskovec, J., Jegelka, S. (2019). How powerful are graph neural networks? *International conference on learning representations.* https://openreview.net/forum?id=ryGs6iA5KmarXiv:1810.00826

Yang, Z., Cohen, W.W., Salakhutdinov, R. (2016). Revisiting semi-supervised learning with graph embeddings. *Proceedings of the 33rd international conference on machine learning* (Vol. 48, pp. 40–48). PMLR. https://proceedings.mlr.press/v48/yanga16.htmlarXiv:1603.08861

Yang, Z.-R., Han, J., Wang, C.-D., & Liu, H. (2025). Erase then rectify: A training-free parameter editing approach for cost-effective graph unlearning. *Proceedings of the aaai conference on artificial intelligence* (Vol. 39, pp. 13044–13051) arXiv:2409.16684.

Zhang, Y., Chen, H., Rida, I., & Zhu, X. (2025). A generative random modality dropout framework for robust multimodal emotion recognition. *IEEE Intelligent Systems, 40*(5), 62–69. https://doi.org/10.1109/MIS.2025.3597120

Zhu, X., Liu, Z., Cambria, E., Yu, X., Fan, X., Chen, H., & Wang, R. (2025). A client-server based recognition system: Non-contact single/multiple emotional and behavioral state assessment methods. *Computer Methods and Programs in Biomedicine, 260*, Article 108564. https://doi.org/10.1016/j.cmpb.2024.108564

Zhu, X., Wang, R., Cascone, L., E, X., & Chen, H. (2026). EUPA: Embodied uncertainty-aware multimodal affect perception for consumer electronic systems. *IEEE Transactions on Consumer Electronics,1–1,*. https://doi.org/10.1109/TCE.2026.3679325

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.